

Telco Customer Churn Classification

A Training-Only Workflow from Data Audit to Supervised Classification

Shakaar Latief

May 28, 2026

Abstract

This document reports the initial data workflow for the Telco Customer Churn classification project. It audits the raw file, validates the target definition, identifies missing or disguised missing values, checks identifier columns, applies deterministic data-quality corrections, constructs a held-out train-test split, and performs exploratory data analysis on the training set only. Feature-target patterns are inspected only after the split, so that modelling decisions are based on the development data while the held-out test set remains reserved for final evaluation.

Contents

1	Raw data audit and deterministic correction	5
1.1	Purpose of the raw audit	5
1.2	Raw dataset overview	5
1.3	Missing-data decision framework	6
1.4	Disguised missing values	6
1.5	Target-label validation	7
1.6	Identifier check	7
1.7	Investigation of <code>TotalCharges</code>	7
1.8	Deterministic interim correction	8
1.9	Basic numeric domain-validity checks	9
1.10	Output of this workflow stage	9
2	Clean modelling dataset and train-test split	11
2.1	Purpose of this stage	11
2.2	Interim dataset	11
2.3	Binary target encoding	11
2.4	Modelling feature set	12
2.5	Semantic feature groups	12
2.6	Final checks before splitting	13
2.7	Target distribution before splitting	13
2.8	Train-test split strategy	13
2.9	Target distribution by split	14
2.10	Saved outputs	14
3	Exploratory data analysis	15
3.1	Purpose of this stage	15
3.2	Training-set overview	15
3.3	Target distribution	15
3.4	Numeric feature summary	16
3.5	Numeric feature distributions	16
3.6	Numeric feature distributions by churn	17
3.7	Numeric scatter matrix	18

3.8	Numeric correlation matrix	19
3.9	Categorical feature frequencies	20
3.10	Categorical churn rates	23
3.11	Selected categorical churn-rate table	25
3.12	Summary	25
4	Preprocessing, evaluation, and simple baselines	26
4.1	Purpose of this stage	26
4.2	Training-only evaluation discipline	26
4.3	Positive and negative classes	27
4.4	Confusion matrix	27
4.5	Accuracy and the class-imbalance problem	28
4.6	Recall, specificity, precision, and F_1	29
4.7	Balanced accuracy	29
4.8	ROC-AUC and PR-AUC	29
4.9	Thresholds and calibration	30
4.10	Class imbalance strategies	30
4.11	Reusable preprocessing infrastructure	31
4.12	Simple baseline definitions	31
4.13	Simple baseline results	32
4.14	Interpretation	33
4.15	Implications for later models	33
5	Linear classification and logistic regression	34
5.1	Purpose of this stage	34
5.2	Linear score functions and decision boundaries	34
5.3	Least-squares classification	34
5.4	Logistic regression	35
5.5	Likelihood and log loss	36
5.6	Regularization	36
5.7	Model comparison	37
5.8	Regularization results	37
5.9	Selected logistic model and coefficient interpretation	39

5.10	Threshold tradeoff	40
5.11	ROC and precision-recall curves	41
5.12	Summary	43
6	k-Nearest Neighbours	44
6.1	k-nearest-neighbour prediction rule	44
6.2	Distance, scaling, and one-hot encoded features	44
6.3	The role of k	45
6.4	Hyperparameter grid	45
6.5	Grid-search behaviour	45
6.6	Model comparison	47
6.7	Threshold behaviour	47
6.8	ROC and precision-recall curves	48
6.9	Summary	50

1 Raw data audit and deterministic correction

1.1 Purpose of the raw audit

This section documents the first workflow stage of the Telco Customer Churn classification project. The purpose is to inspect the raw dataset before constructing the modelling table and before creating the held-out train-test split.

The raw audit is deliberately limited. It does not estimate predictive patterns, compare feature distributions by churn status, create churn-rate plots, select features, tune preprocessing, or fit models. Instead, it answers the following questions:

- Does the raw file load correctly?
- What are the raw columns and data types?
- Are there duplicate rows?
- Are there standard missing values?
- Are there disguised missing values, such as blank strings?
- Does the target column exist and contain valid labels?
- Is there an identifier column that should be excluded from modelling?
- Are there deterministic raw-data representation problems that must be corrected before splitting?

This separation is important for leakage control. The full raw file may be inspected for schema and data-quality problems, because a valid modelling dataset cannot be created without those checks. However, feature-target exploratory analysis and any data-dependent modelling choices are postponed until after the train-test split and will use the training set only.

1.2 Raw dataset overview

The raw dataset contains 7043 observations and 21 columns. No exact duplicate rows were found. Pandas did not detect any standard missing values.

Table 1: Raw dataset overview

Item	Value
Number of rows	7043
Number of columns	21
Duplicate rows	0
Total standard missing values	0
Columns with standard missing values	0

The absence of standard missing values is not enough to conclude that no information is missing. String columns may contain blank strings or placeholder values that are not detected by ordinary missing-value checks.

1.3 Missing-data decision framework

Before choosing a cleaning strategy, it is useful to distinguish two different missing-data problems:

1. Missing feature values: at least one input variable X is missing.
2. Missing target labels: the output variable Y is missing.

These two cases should not be handled in the same way. For missing feature values, the central production question is whether missing inputs may occur when the model is used in the future. If missing feature values may occur in production, the final model pipeline must be able to handle them. In that setting, missing values should remain present in validation and test data so that evaluation simulates production. Any imputation rule or preprocessing rule must be fitted on the training data only and then applied unchanged to validation, test, and production data.

If production data is expected to be clean but the training data contains missing feature values, the goal is different: the experiment should simulate clean production. In that case, possible training-data strategies include removing incomplete training rows, imputing training values, removing a feature, or using model-based imputation. The choice should be made using training and validation logic, not the held-out test set.

For missing target labels, the rules are different. Missing labels in the training set may require training only on labelled observations, label imputation, or semi-supervised learning. Missing labels in the test set should not simply be imputed and treated as known. If test labels are missing, performance should be reported with uncertainty, for example through best-case and worst-case bounds.

In this dataset, the raw audit found no missing target labels and no standard missing feature values. It did find a disguised missing or representation issue in **TotalCharges**, which is discussed below.

1.4 Disguised missing values

The string-column audit checks for blank strings after trimming whitespace. This is necessary because blank strings such as "" or " " are not automatically treated as missing values by pandas.

Table 2: Blank-string audit for selected string-like columns

Column	Blank string count	Blank string percentage	Unique values
TotalCharges	11	0.16	6531
customerID	0	0.00	7043
PaymentMethod	0	0.00	4
MultipleLines	0	0.00	3
InternetService	0	0.00	3
OnlineSecurity	0	0.00	3
OnlineBackup	0	0.00	3

Only **TotalCharges** contains blank strings. This is important because **TotalCharges** is conceptually numeric, but the blank strings cause the raw column to be read as string-like.

1.5 Target-label validation

The target variable is **Churn**. The observed target labels are **No** and **Yes**. There are no missing target labels and no unexpected target labels.

Table 3: Target distribution in the raw dataset

Target level	Count	Percentage
No	5174	73.46
Yes	1869	26.54

The positive class is defined as **Churn = Yes**. The target distribution is moderately imbalanced, with approximately 26.54% churn. This check justifies using a stratified train-test split in the next workflow stage. It is not used for model selection.

Table 4: Target-label validity check

Item	Value
Expected negative label present	True
Expected positive label present	True
Missing target values	0
Unexpected target labels	None

1.6 Identifier check

The column **customerID** has 7043 unique values for 7043 rows. It is therefore a unique identifier candidate. The identifier is useful for traceability, but it should not be used as a modelling feature because it does not describe a generalizable customer characteristic.

Table 5: Identifier check

Column	Unique values	Number of rows	Unique value ratio
customerID	7043	7043	1.00
TotalCharges	6531	7043	0.93
MonthlyCharges	1585	7043	0.23
tenure	73	7043	0.01
PaymentMethod	4	7043	0.00

1.7 Investigation of TotalCharges

The blank-string audit showed 11 blank strings in **TotalCharges**. These rows were checked against **tenure**. The result is exact: the 11 blank **TotalCharges** rows are exactly the 11 rows where **tenure = 0**.

Table 6: Relationship between blank **TotalCharges** and zero tenure

Condition	Count
Blank TotalCharges	11
tenure == 0	11
Blank TotalCharges and tenure == 0	11
Blank TotalCharges and tenure != 0	0
Non-blank TotalCharges and tenure == 0	0

Converting **TotalCharges** to numeric creates exactly 11 missing values, which are the same 11 blank-string records.

Table 7: **TotalCharges** numeric conversion check

Item	Count
Standard missing before conversion	0
Missing after numeric conversion	11
New missing created by numeric conversion	11
Blank string count	11

The affected rows are all zero-tenure customers. Most of these records have a two-year contract and none of them churned in the raw data. The churn outcome is not used to justify the correction. The correction is justified by the meaning of **TotalCharges**: a zero-tenure customer has not accumulated charges yet.

1.8 Deterministic interim correction

The following correction is applied to an interim copy of the raw dataset:

1. Convert **TotalCharges** from string-like values to numeric values.
2. For rows where **tenure** = 0 and **TotalCharges** is blank after conversion, set **TotalCharges** = 0.0.

This is not mean imputation, median imputation, model-based imputation, or a target-informed correction. It is a deterministic representation correction based on the definition of accumulated charges. The original raw dataframe is kept unchanged, and the corrected copy is saved as the interim dataset for the next workflow stage.

After the correction, **TotalCharges** is numeric, contains no missing values, and ranges from 0.0 to 8684.8.

Table 8: Post-correction check

Item	Value
Remaining total missing values	0
Remaining missing TotalCharges	0
TotalCharges dtype	float64
Minimum TotalCharges	0.00
Maximum TotalCharges	8684.80

1.9 Basic numeric domain-validity checks

Unusual values are not automatically errors. A high but possible value may be a natural observation that belongs to the production distribution. However, values that violate basic domain constraints are different: they may indicate coding errors, measurement errors, or corrupted records.

The three numeric variables in this dataset should be non-negative:

- **tenure** is the number of months a customer has stayed with the company;
- **MonthlyCharges** is the customer’s monthly charge amount;
- **TotalCharges** is the accumulated charge amount.

A value of zero can be valid. For example, zero tenure occurs for newly registered customers, and the corresponding accumulated total charge can be zero. Therefore, this audit checks only for negative values. This is not statistical outlier detection. Statistical outlier analysis is postponed until training-set exploratory analysis.

Table 9: Numeric domain-validity check after interim correction

Feature	Minimum	Maximum	Negative count	Negative percentage
tenure	0.00	72.00	0	0.00
MonthlyCharges	18.25	118.75	0	0.00
TotalCharges	0.00	8684.80	0	0.00

No negative values were found in the numeric variables. Therefore, no additional corrupted numeric values are corrected in this raw-audit stage.

1.10 Output of this workflow stage

The raw audit produces an interim dataset with 7043 rows, 21 columns, and no missing values. The raw target column **Churn** remains unchanged. The next workflow stage will create the binary target **Churn_binary**, exclude **customerID** from the modelling feature set, and create the held-out stratified train-test split.

Table 10: Interim dataset save check

Item	Value
Number of rows	7043
Number of columns	21
Total missing values	0
Interim file	<code>data/interim/telco_churn_interim.csv</code>

2 Clean modelling dataset and train-test split

2.1 Purpose of this stage

The raw-data audit produced an interim dataset in which the representation problem in **TotalCharges** was corrected. The next step is to create the supervised modelling table and the held-out train-test split.

This stage is deliberately narrow. It creates the binary target, defines the feature set, validates semantic feature groups, and creates a stratified split. It does not perform exploratory feature-target analysis, one-hot encoding, scaling, feature engineering, resampling, feature selection, or model fitting. Those steps are postponed until after the split and should use the training set only.

2.2 Interim dataset

The interim dataset is loaded from:

```
data/interim/telco_churn_interim.csv.
```

It has the same raw columns as the original dataset, except that **TotalCharges** has been converted to a numeric representation and the zero-tenure blank values have been set to 0.0. No remaining missing values are present at this stage.

Table 11: Interim dataset overview

Item	Value
Number of rows	7043
Number of columns	21
Duplicate rows	0
Total missing values	0
Columns with missing values	0

2.3 Binary target encoding

The supervised learning target is **Churn_binary**. The positive class is churn, because this is the event of practical interest.

$$\text{Churn_binary} = \begin{cases} 1, & \text{if Churn} = \text{Yes}, \\ 0, & \text{if Churn} = \text{No}. \end{cases}$$

The encoding produces no missing values and preserves the two-class structure of the original target.

Table 12: Target encoding

Original target level	Encoded value
No	0
Yes	1

2.4 Modelling feature set

The modelling feature set excludes `customerID`, the original string target `Churn`, and the binary target `Churn.binary`. The identifier is excluded because it is row-specific and does not represent a generalizable customer characteristic. The original target is excluded because the model uses the encoded binary target.

The resulting modelling table contains 7043 observations, 19 feature columns, and one target column.

Table 13: Clean modelling table overview

Item	Value
Rows	7043
Feature columns	19
Target columns	1
Identifier included as feature	No
Original string target included as feature	No

2.5 Semantic feature groups

The 19 modelling features are divided into semantic groups. These groups describe the clean base dataset; they do not yet define the final preprocessing for every model. Model-specific preprocessing is handled later inside training-only pipelines.

Table 14: Feature groups for modelling

Feature group	Count	Features
Numeric features	3	<code>tenure</code> , <code>MonthlyCharges</code> , <code>TotalCharges</code>
Binary categorical features	6	<code>SeniorCitizen</code> , <code>gender</code> , <code>Partner</code> , <code>Dependents</code> , <code>PhoneService</code> , <code>PaperlessBilling</code>
Nominal categorical features	10	<code>MultipleLines</code> , <code>InternetService</code> , <code>OnlineSecurity</code> , <code>OnlineBackup</code> , <code>DeviceProtection</code> , <code>TechSupport</code> , <code>StreamingTV</code> , <code>StreamingMovies</code> , <code>Contract</code> , <code>PaymentMethod</code>

The feature-group validation confirms that every modelling feature belongs to exactly one group: no feature is missing from the groups, no grouped feature lies outside the modelling feature set, and no feature is assigned more than once.

2.6 Final checks before splitting

The final modelling table contains no missing values. Exact duplicate rows in the full interim data are absent. After excluding the identifier, duplicate feature-target combinations may exist, but this is not automatically a data-quality issue because multiple customers can genuinely share the same observed characteristics and churn label.

Table 15: Final checks before splitting

Item	Value
Missing values in modelling table	0
Duplicate full interim rows	0
Duplicate feature-target rows after excluding identifier	22

2.7 Target distribution before splitting

The target distribution is moderately imbalanced. The positive class, churn, represents approximately 26.5% of observations. This motivates a stratified train-test split. This target-only summary is used for split design rather than feature selection or model interpretation. No feature-target relationships are inspected before the train-test split.

Table 16: Target distribution before splitting

Churn_binary	Count	Percentage
0	5174	73.46
1	1869	26.54

2.8 Train-test split strategy

The clean modelling table is split into a training set and a held-out test set. The test set is reserved for final evaluation and should not be used for feature-target EDA, preprocessing design, model selection, hyperparameter tuning, or threshold tuning.

The split uses:

```
test_size = 0.20,    random_state = 42.
```

The split is stratified by `Churn_binary`, so the train and test sets preserve the original churn proportion.

Table 17: Train-test split overview

Split	Rows	Percentage
Train	5634	79.99
Test	1409	20.01

2.9 Target distribution by split

The stratified split preserves the target distribution closely.

Table 18: Target distribution by split

Split	Churn_binary	Count	Percentage
Train	0	4139	73.46
Train	1	1495	26.54
Test	0	1035	73.46
Test	1	374	26.54

2.10 Saved outputs

The training and test datasets are saved locally as:

`data/processed/train.csv`, `data/processed/test.csv`.

These files form the boundary for the rest of the project. The next workflow stage should load only `train.csv` for exploratory analysis and modelling decisions. The held-out test file should remain unused until final model evaluation.

3 Exploratory data analysis

3.1 Purpose of this stage

The raw-data audit and splitting stages created a corrected interim dataset, a clean modelling table, and a held-out train-test split. This section performs exploratory data analysis using only the training set.

This restriction is methodological. Exploratory findings can influence later modelling choices, including feature engineering, transformations, rule-based baselines, model selection, outlier treatment, and interpretation. Therefore, feature distributions and feature-target relationships are not inspected on the held-out test set.

The goals of this stage are to understand the training data, identify modelling-relevant patterns, and produce report-ready figures and tables while preserving the test set for final evaluation. Large figures are split when needed for readability; a figure is kept in the main text when it is directly used for interpretation, while purely supplementary diagnostics can be moved to an appendix in later stages.

3.2 Training-set overview

The dataset used in this section contains 5634 observations. It has 19 feature columns and the binary target column `Churn_binary`. No missing values remain after the raw audit and deterministic `TotalCharges` correction.

Table 19: Training-set overview

Item	Value
Training rows	5634
Training columns	20
Missing values	0
Columns with missing values	0
Duplicate feature-target rows	15

The duplicate feature-target rows are not treated as a data-quality issue. The unique customer identifier was intentionally removed before modelling, so different customers can legitimately share the same observed feature values and churn label.

3.3 Target distribution

The positive class is churn, encoded as 1. The target distribution remains moderately imbalanced, with churn representing 26.54% of observations.

Table 20: Target distribution

<code>Churn_binary</code>	Count	Percentage
0	4139	73.46
1	1495	26.54

This imbalance is not extreme, but it is large enough that accuracy alone will be an incomplete evaluation metric. Later modelling sections should include metrics that distinguish false positives from false negatives, such as precision, recall, specificity, F1-score, ROC-AUC, PR-AUC, and threshold analysis.

3.4 Numeric feature summary

The dataset has three numeric features: `tenure`, `MonthlyCharges`, and `TotalCharges`. Numeric summaries are computed on this development split only.

Table 21: Numeric feature summary

Feature	Count	Mean	Std.	Min	25%	Median	75%	Max
<code>tenure</code>	5634	32.49	24.57	0.00	9.00	29.00	55.00	72.00
<code>MonthlyCharges</code>	5634	64.93	30.14	18.40	35.66	70.50	90.00	118.75
<code>TotalCharges</code>	5634	2299.33	2279.20	0.00	402.98	1394.93	3835.83	8684.80

The numeric variables have different scales. This matters for later model families such as k-nearest neighbours, support vector machines, logistic regression with regularization, and neural networks, where scaling is usually required. Scaling should be fitted inside training-only pipelines rather than on the full dataset.

3.5 Numeric feature distributions

Figure 1 shows histograms and boxplots for the three numeric features in this split.

Numeric Feature Distributions

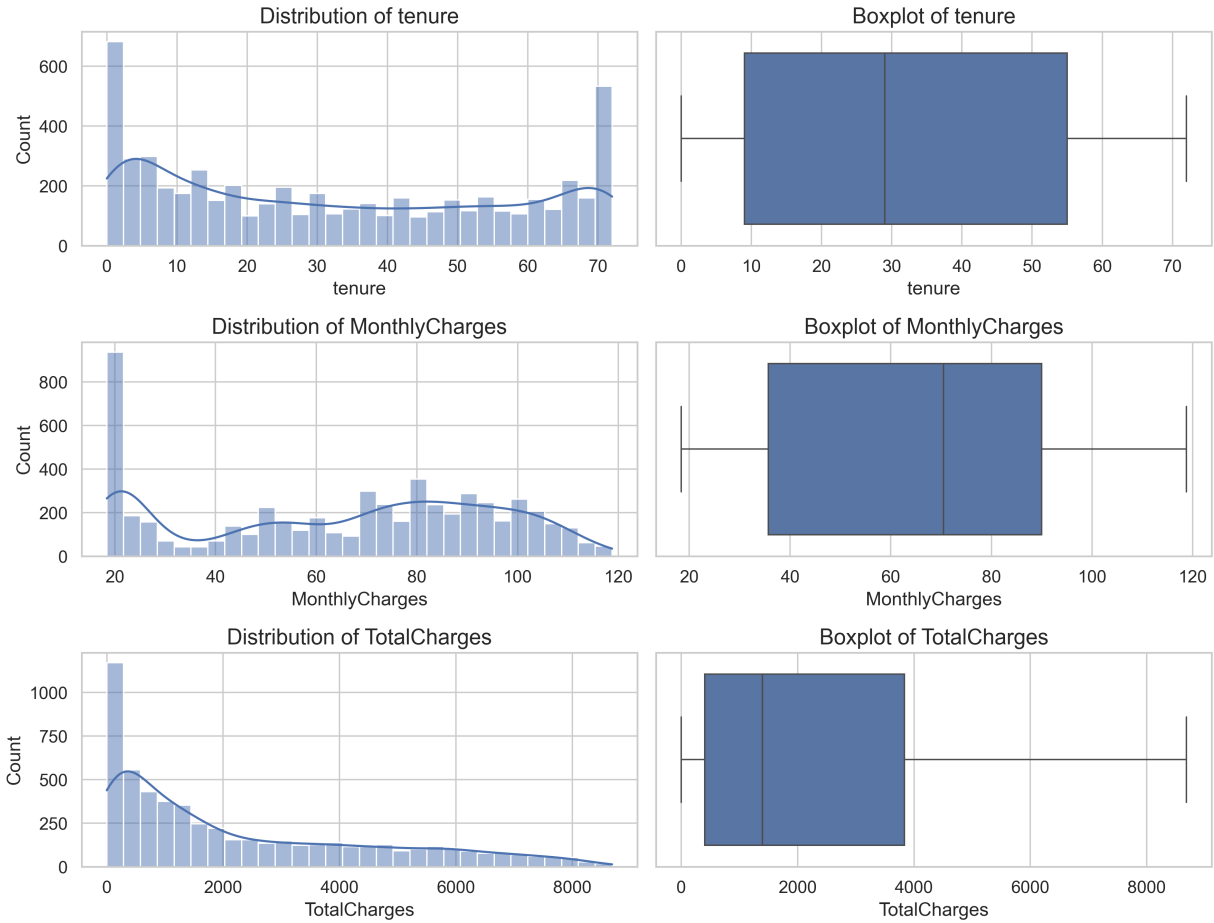


Figure 1: Numeric feature distributions and boxplots

The distribution of **tenure** has substantial mass near low values and also many observations near the upper end of the observed range. This indicates that the training set contains both newly joined customers and long-tenure customers.

The distribution of **MonthlyCharges** is not symmetric. There is a large group of customers with relatively low monthly charges and a broad group at higher monthly charge levels. The distribution of **TotalCharges** is strongly right-skewed. This is expected because total charges accumulate over time: many customers have low accumulated charges, while long-tenure or high-monthly-charge customers can have much larger accumulated totals.

These plots are descriptive. They do not justify automatic outlier removal. The raw audit already checked for impossible negative numeric values. High but plausible values should be treated as potential natural observations unless there is separate evidence that they are coding errors.

3.6 Numeric feature distributions by churn

Figure 2 compares the numeric feature distributions between churners and non-churners.

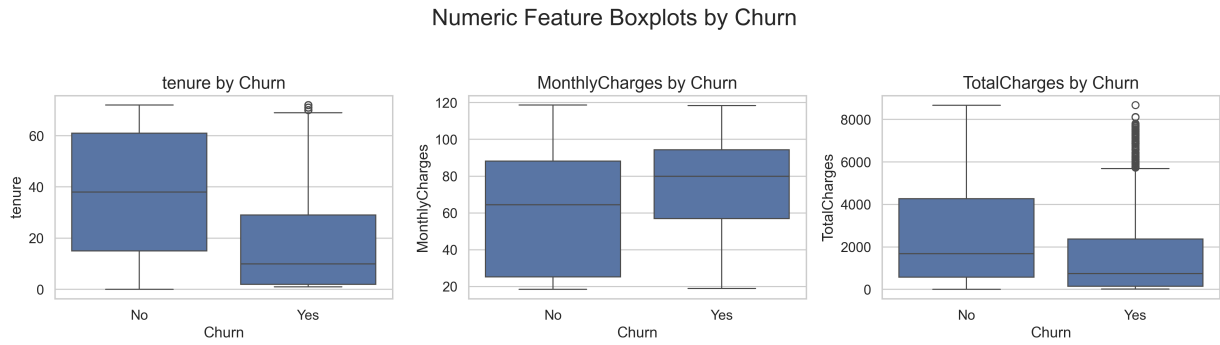


Figure 2: Numeric feature boxplots by churn

The class-conditioned boxplots show clearer feature-target structure than the marginal distributions. Customers who churned have substantially lower **tenure** than customers who did not churn. This is one of the strongest numeric patterns in the training data and suggests that newer customers are more likely to leave.

MonthlyCharges tends to be higher for churners. This indicates that customers with higher monthly bills may be at greater churn risk, at least as a marginal training-set association. **TotalCharges** tends to be lower for churners, but this should be interpreted carefully. Since **TotalCharges** accumulates over time, lower total charges among churners are consistent with churners having shorter tenure.

3.7 Numeric scatter matrix

Figure 3 shows a scatter matrix for the numeric features, coloured by churn status.

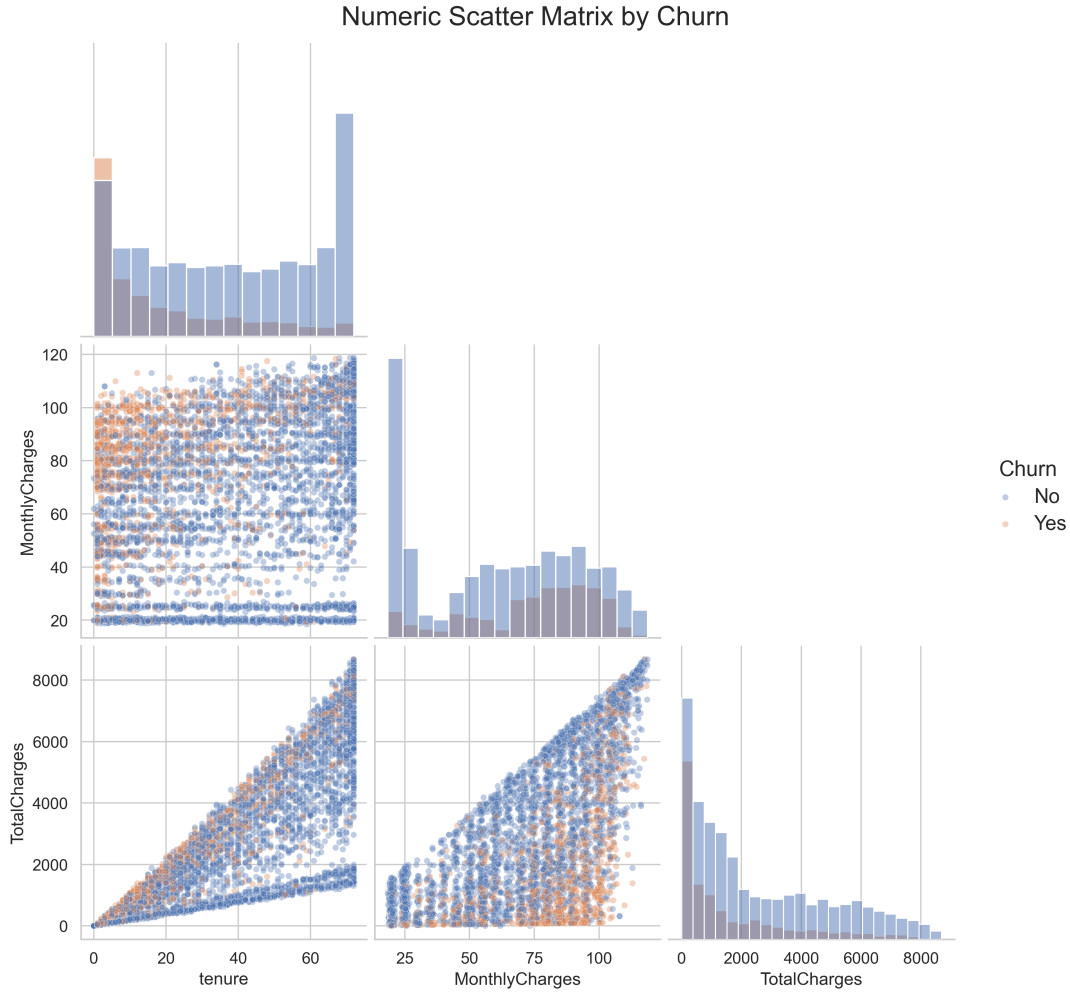


Figure 3: Numeric scatter matrix by churn

The scatter matrix shows visible churn-related patterns, but it does not show a clean visual separation between churners and non-churners using any single pair of numeric variables. Churners are more concentrated among customers with shorter tenure and are also relatively visible among customers with higher monthly charges. However, the two classes overlap substantially across the numeric feature space, suggesting that churn is not separable by a simple pairwise numeric boundary.

The clearest structural pattern is the relationship between `tenure` and `TotalCharges`. The triangular shape is expected because total charges accumulate over months and are also affected by monthly charges. For a fixed tenure, customers with higher monthly charges can accumulate higher total charges. This motivates using models that can combine numeric and categorical information, and possibly models that can learn interactions.

3.8 Numeric correlation matrix

Figure 4 shows the Pearson correlation matrix among numeric features and `Churn_binary` in this split.

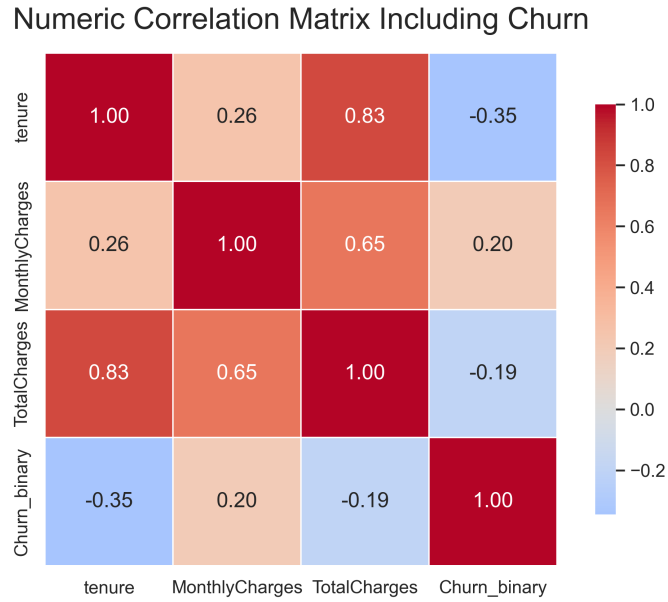


Figure 4: Numeric correlation matrix including **Churn_binary**

The strongest numeric correlation with churn is for **tenure**, with correlation approximately -0.35 . Longer-tenure customers are less likely to churn in the training data. **MonthlyCharges** has a positive correlation with churn of approximately 0.20, while **TotalCharges** has a negative correlation with churn of approximately -0.19 .

The correlation matrix also shows that **tenure** and **TotalCharges** are strongly correlated, approximately 0.83, and that **MonthlyCharges** and **TotalCharges** are also positively correlated, approximately 0.65. These are marginal linear associations only. They do not capture nonlinear relationships, interactions, or the effects of categorical variables.

3.9 Categorical feature frequencies

Figures 5 and 6 show the distribution of categorical feature levels. The categorical variables are split across two figures to keep axis labels and category names readable in the compiled report.

Categorical Feature Distributions, Part I

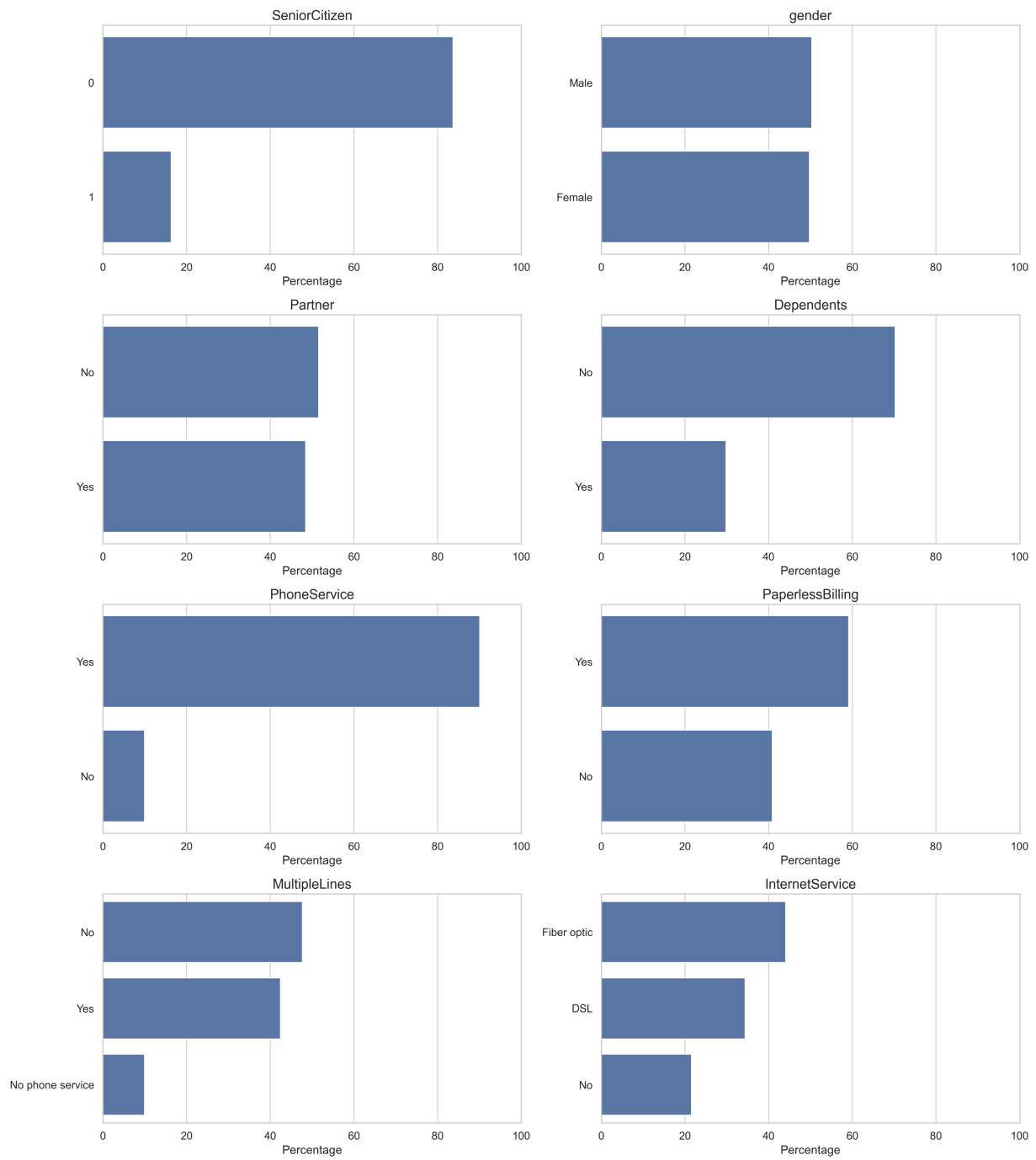


Figure 5: Categorical feature distributions, Part I

Categorical Feature Distributions, Part II

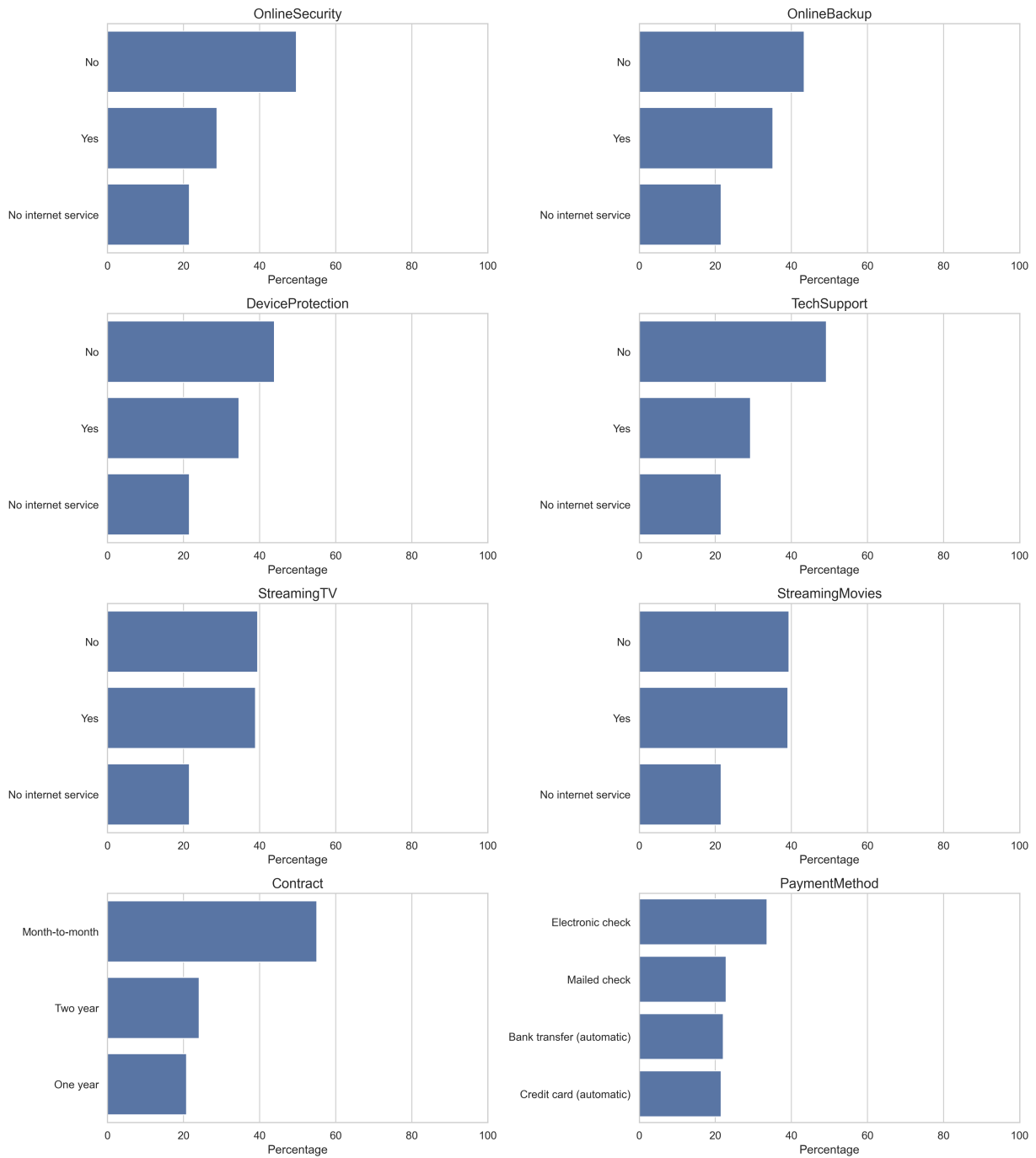


Figure 6: Categorical feature distributions, Part II

Several categorical predictors contain strongly imbalanced levels. For example, `PhoneService = Yes` represents 90.08% of the training set, `SeniorCitizen = 0` represents 83.67%, and `Contract = Month-to-month` represents 55.06%. These imbalances matter because rare levels can create unstable estimates in simple baselines and can affect how much information a model can learn from a category.

Several internet-service add-on variables contain the structural level `No internet service`. This is not missingness. It is an informative category indicating that the customer does not have internet service. Later preprocessing should preserve this category rather than treating it as a

missing value.

3.10 Categorical churn rates

Figures 7 and 8 show the churn rate within each categorical level. Both parts are included in the main section because they support the interpretation: Part I contains demographic, household, billing, phone-service, and internet-service variables, while Part II contains several service add-ons as well as contract type and payment method.

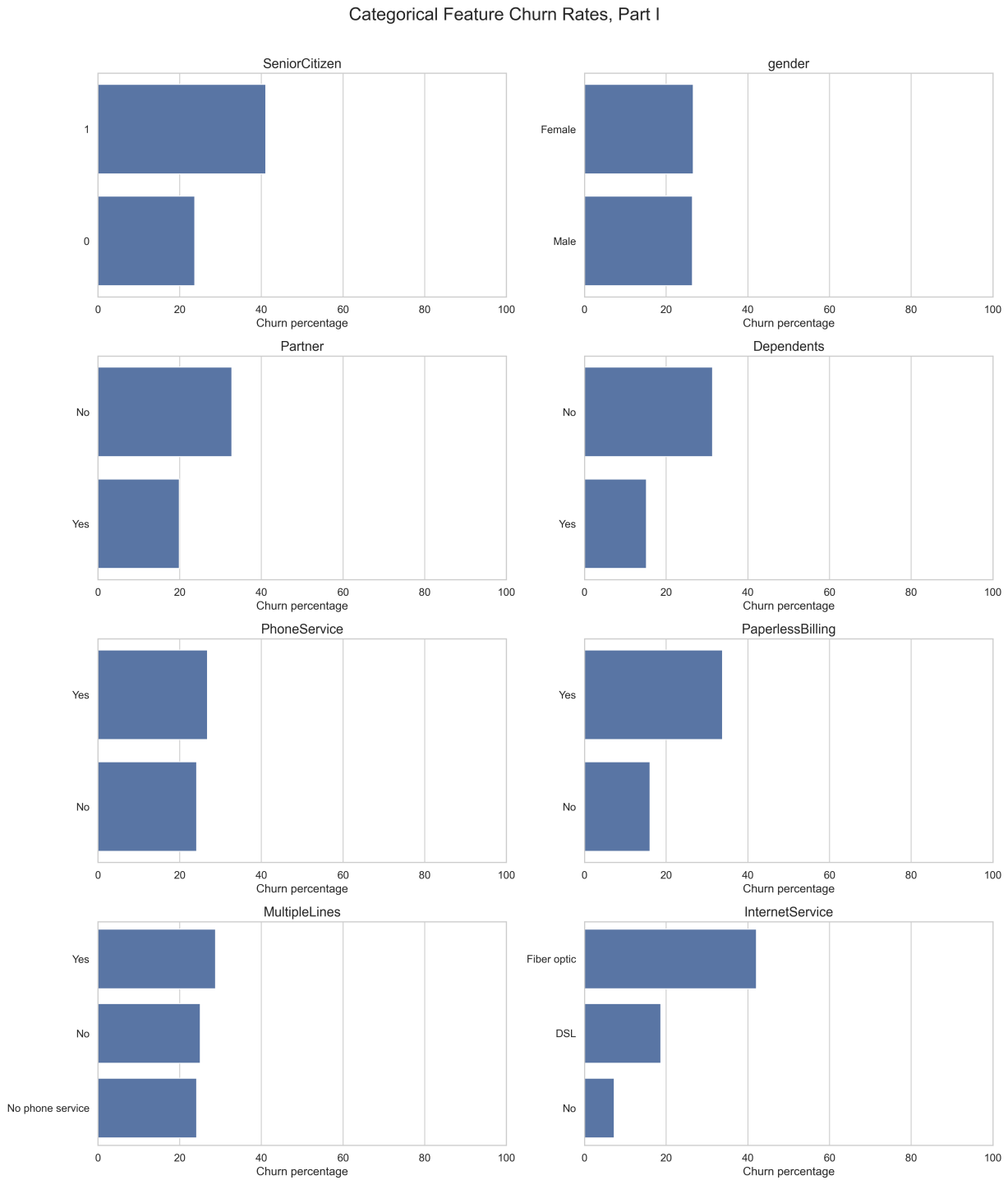


Figure 7: Categorical feature churn rates, Part I

Categorical Feature Churn Rates, Part II

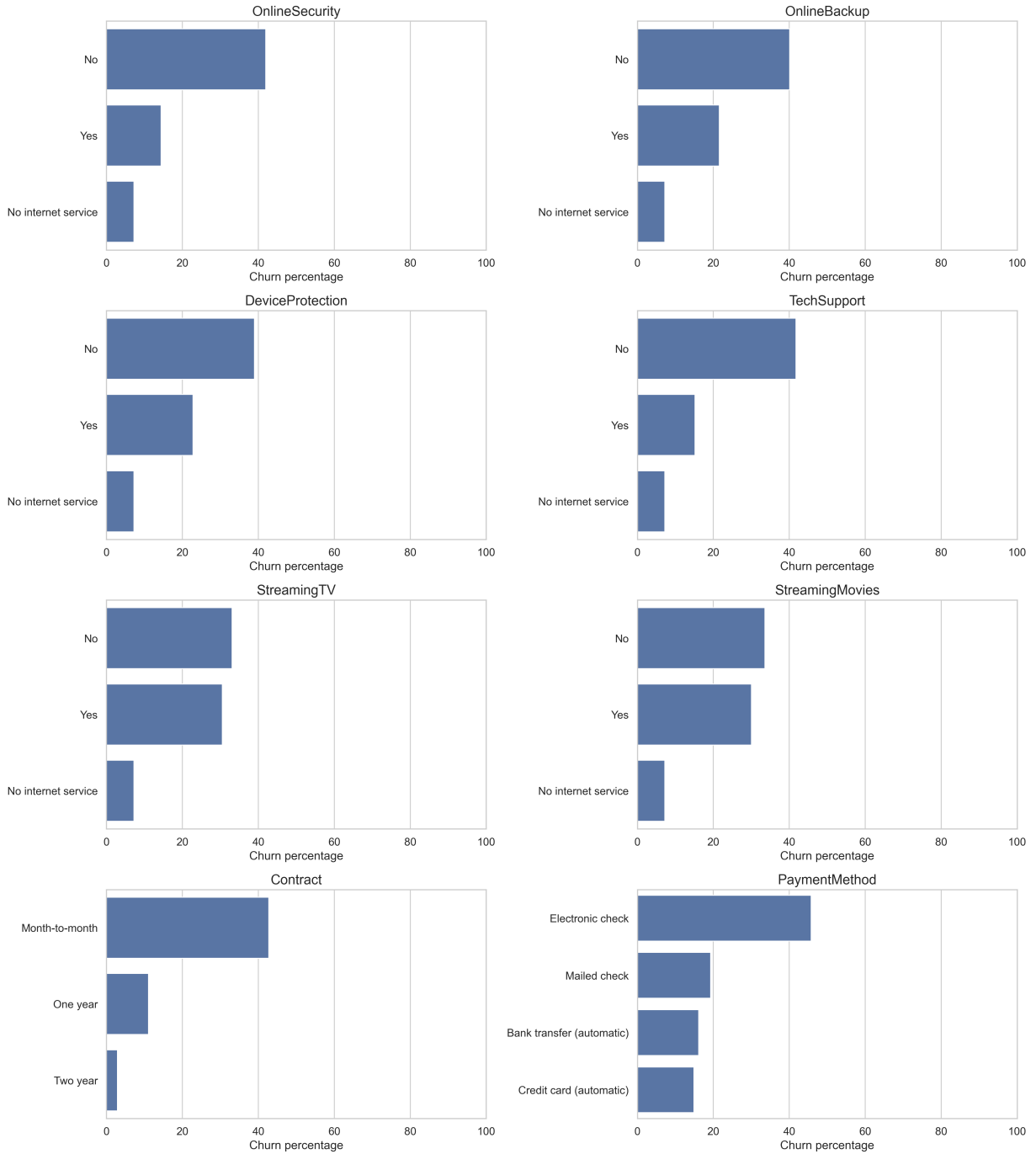


Figure 8: Categorical feature churn rates, Part II

The categorical churn-rate plots reveal several large training-set differences. Contract type is one of the strongest categorical patterns: month-to-month customers have a churn rate of 42.75%, compared with 11.08% for one-year contracts and 2.87% for two-year contracts. Internet service type also differs strongly: fiber optic customers have a churn rate of 42.09%, compared with 18.69% for DSL and 7.25% for customers with no internet service.

Payment method is also informative. Customers using electronic check have a churn rate of 45.74%, higher than mailed check at 19.28%, bank transfer at 16.16%, and credit card at 14.92%. Customers without online security or tech support also show much higher churn rates than

customers with those services.

Some variables show weak marginal separation. For example, churn rates for **gender** are almost identical, and **PhoneService** shows only a small difference. This does not mean these variables must be removed, because they may still contribute through interactions or in combination with other features. It does suggest that their standalone marginal relationship with churn is weak.

3.11 Selected categorical churn-rate table

Table 22 reports selected categorical levels that are especially relevant for interpretation and later baseline design.

Table 22: Selected categorical churn rates

Feature	Level	Count	Training %	Churn %
Contract	Month-to-month	3102	55.06	42.75
Contract	One year	1173	20.82	11.08
Contract	Two year	1359	24.12	2.87
InternetService	Fiber optic	2483	44.07	42.09
InternetService	DSL	1937	34.38	18.69
InternetService	No	1214	21.55	7.25
PaymentMethod	Electronic check	1891	33.56	45.74
PaperlessBilling	Yes	3331	59.12	33.80
PaperlessBilling	No	2303	40.88	16.02
SeniorCitizen	1	920	16.33	41.09
SeniorCitizen	0	4714	83.67	23.70

These are associations in the training data, not causal effects. They are useful for understanding the dataset, designing simple baselines, and interpreting later models.

3.12 Summary

The EDA suggests that the dataset contains meaningful predictive structure. Numerically, churn is most clearly associated with lower tenure and somewhat higher monthly charges. **TotalCharges** is lower among churners, but this is closely related to the shorter tenure of churned customers.

Categorically, the strongest visible churn-rate differences occur for contract type, internet service type, payment method, online security, tech support, paperless billing, and senior-citizen status. These patterns motivate the next stage: building preprocessing pipelines and baseline classifiers using only the training set.

4 Preprocessing, evaluation, and simple baselines

4.1 Purpose of this stage

This section starts the modelling workflow after the raw-data audit, deterministic correction, train/test split, and training-set exploratory analysis. The purpose is not yet to fit flexible learned models. Instead, this stage establishes the evaluation language and simple reference baselines that all later models must improve upon.

The held-out test set remains unused. All results in this section are computed only from the training set using stratified cross-validation. This preserves the final test set for the final evaluation stage.

The section has three goals. First, it defines the binary classification evaluation framework used throughout the rest of the project. Second, it introduces reusable preprocessing infrastructure for later models. Third, it evaluates simple baselines that do not learn a flexible relationship from the full feature matrix.

The simple baselines considered here are:

1. a majority-class baseline;
2. a prior-probability baseline;
3. a stratified random baseline;
4. a uniform random baseline;
5. an exploratory-analysis-inspired rule baseline.

Learned models such as least-squares classification, logistic regression, k-nearest neighbours, Naive Bayes, decision trees, support vector machines, and neural networks are introduced in later sections.

4.2 Training-only evaluation discipline

Let the training set be

$$\mathcal{D}_{train} = \{(x_i, y_i)\}_{i=1}^n,$$

where x_i denotes the feature vector for customer i , and $y_i \in \{0, 1\}$ denotes the binary churn label. In this project, the positive class is churn:

$$y_i = 1 \iff \text{customer } i \text{ churned.}$$

All model-development decisions must be made without using the held-out test set. This includes preprocessing design, feature engineering, feature selection, model-family choice, hyperparameter tuning, threshold tuning, calibration decisions, and model comparison.

For this stage, performance is estimated using stratified 5-fold cross-validation inside the training set. Stratification preserves approximately the same churn rate in each fold. This is important because churn is the minority class. In the training set, the class distribution is shown in Table 23.

Table 23: Training-set target distribution.

Class	Count	Percentage
No churn ($Y = 0$)	4139	73.46%
Churn ($Y = 1$)	1495	26.54%

Cross-validation produces out-of-fold predictions. This means that the prediction for each observation is generated by a model that was not fitted on that same observation. This gives a more honest development-stage estimate than evaluating a model on the same data used to fit it.

4.3 Positive and negative classes

Many binary classification metrics are defined relative to the positive class. The positive class is not necessarily the most common class. It is the event of interest.

A medical screening example makes the terminology intuitive. Suppose a test predicts whether a person has cancer. Then:

- positive class: the person has cancer;
- negative class: the person does not have cancer;
- predicted positive: the test says cancer;
- predicted negative: the test says no cancer.

For the churn problem, the same logic becomes:

- positive class: the customer churns;
- negative class: the customer does not churn;
- predicted positive: the model flags the customer as likely to churn;
- predicted negative: the model does not flag the customer as likely to churn.

This convention matters because recall, precision, F_1 , ROC-AUC, and PR-AUC all depend on which class is treated as positive.

4.4 Confusion matrix

The confusion matrix counts the four possible combinations of actual class and predicted class. The positive-first layout used here is shown in Table 24. Rows represent the actual class and columns represent the predicted class.

Table 24: Positive-first binary confusion matrix layout.

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
Actual	Negative	FP	TN

The entries are:

$$\begin{aligned} TP &= \#\{\hat{y} = 1, y = 1\}, & FN &= \#\{\hat{y} = 0, y = 1\}, \\ FP &= \#\{\hat{y} = 1, y = 0\}, & TN &= \#\{\hat{y} = 0, y = 0\}. \end{aligned}$$

This order is chosen to match the lecture-style visual explanation: actual positives are shown first, and within that row the model can either correctly predict positive or incorrectly predict negative. Some software libraries store confusion matrices in a negative-first numeric order. In the code, the counts are computed with explicit labels and then renamed so that the report can present the matrix in the positive-first order.

In the medical screening example:

- true positive: the test says cancer, and the person really has cancer;
- false negative: the test says no cancer, but the person really has cancer;
- false positive: the test says cancer, but the person does not have cancer;
- true negative: the test says no cancer, and the person does not have cancer.

In the churn problem:

- true positive: the model predicts churn, and the customer churns;
- false negative: the model predicts no churn, but the customer churns;
- false positive: the model predicts churn, but the customer does not churn;
- true negative: the model predicts no churn, and the customer does not churn.

False positives and false negatives have different practical meanings. A false positive may lead to unnecessary retention effort for a customer who would have stayed anyway. A false negative may mean missing a customer who actually leaves.

4.5 Accuracy and the class-imbalance problem

Accuracy is the fraction of all predictions that are correct:

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN}.$$

Accuracy is intuitive, but it can be misleading when one class is much more common than the other. In this dataset, about 73.46% of training customers do not churn. A classifier that always predicts no churn can therefore achieve 73.46% accuracy while detecting zero churners.

This is the central reason why the project does not evaluate churn models using accuracy alone. The evaluation must also measure how well the model detects the positive class and how many false churn alerts it creates.

4.6 Recall, specificity, precision, and F_1

Recall, also called true positive rate, measures the fraction of actual positives that are detected:

$$\text{Recall} = \text{TPR} = \frac{TP}{TP + FN}.$$

In this project, recall answers: among all customers who churned, how many did the model flag?

Specificity, also called true negative rate, measures the fraction of actual negatives correctly rejected:

$$\text{Specificity} = \text{TNR} = \frac{TN}{TN + FP}.$$

In this project, specificity answers: among all customers who did not churn, how many did the model correctly leave unflagged?

Precision measures the reliability of positive predictions:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

In this project, precision answers: among all customers flagged as churners, how many actually churned?

The F_1 -score combines precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

It is useful as a compact summary when both precision and recall matter. However, it hides the separate business meanings of false positives and false negatives. For this reason, F_1 is reported together with the underlying precision and recall values.

4.7 Balanced accuracy

Balanced accuracy averages recall and specificity:

$$\text{Balanced Accuracy} = \frac{1}{2} (\text{Recall} + \text{Specificity}).$$

Balanced accuracy is useful under class imbalance because it gives equal importance to the positive and negative classes. A classifier that always predicts one class receives balanced accuracy equal to 0.5 in a binary problem, because it perfectly classifies one class and completely fails on the other.

This makes balanced accuracy useful for comparing simple baselines. A majority-class classifier may have high ordinary accuracy, but its balanced accuracy reveals whether it is actually useful for both classes.

4.8 ROC-AUC and PR-AUC

Many classifiers produce a score or probability rather than only a hard class label. A threshold turns this score into a class prediction:

$$\hat{y}_\tau = \mathbb{I}\{s(x) \geq \tau\},$$

where $s(x)$ is a model score or predicted probability and τ is the decision threshold.

Changing the threshold changes the tradeoff between false positives and false negatives. ROC and precision-recall curves summarize this threshold-dependent behaviour.

The ROC curve plots:

$$\text{FPR} = \frac{FP}{FP + TN} \quad \text{against} \quad \text{TPR} = \frac{TP}{TP + FN}.$$

ROC-AUC summarizes how well the model ranks positive cases above negative cases across thresholds.

The precision-recall curve plots:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{against} \quad \text{Recall} = \frac{TP}{TP + FN}.$$

PR-AUC summarizes how precise the positive predictions remain as the model attempts to recover more positives.

ROC-AUC and PR-AUC often move in the same direction when one model is clearly better at ranking positive cases. They can differ under class imbalance. A model can have a reasonable ROC-AUC while still having low precision if many of its predicted positives are false positives. Since churn is the minority class and the positive class is the main event of interest, PR-AUC is an important complement to ROC-AUC.

4.9 Thresholds and calibration

A probabilistic classifier estimates:

$$\hat{p}(Y = 1 \mid X = x).$$

The default hard classification rule is often:

$$\hat{y} = \mathbb{I}\{\hat{p}(Y = 1 \mid X = x) \geq 0.5\}.$$

The threshold 0.5 is not automatically optimal. In churn prediction, the preferred threshold depends on the relative cost of offering retention actions to customers who would not churn versus missing customers who will churn.

Calibration is a separate question. A model is calibrated if predicted probabilities behave like empirical probabilities:

$$P(Y = 1 \mid \hat{p} = q) \approx q.$$

For example, among customers assigned a churn probability near 0.30, about 30% should actually churn. Calibration will become more important when learned probability models are introduced.

4.10 Class imbalance strategies

The current section evaluates baselines under the natural training distribution. Resampling strategies such as random oversampling, random undersampling, and SMOTE are not applied yet because they are training interventions for learned models rather than baseline definitions.

The validation and test distributions should remain realistic. If resampling is used later, it must be applied only inside the training part of each cross-validation split. The validation fold must remain untouched. The incorrect workflow is:

resample full training set $\rightarrow$ cross-validate on resampled data.

This can leak information or artificially improve validation performance because the validation fold no longer represents an untouched natural sample.

The correct workflow is:

split fold $\rightarrow$ fit preprocessing on training fold $\rightarrow$ resample training fold only $\rightarrow$ fit model $\rightarrow$ c

In Python, this usually requires an imbalanced-learn pipeline rather than an ordinary scikit-learn pipeline, because the sampler changes both X and y . Resampling, class weighting, and threshold tuning are considered later after learned models have been introduced.

4.11 Reusable preprocessing infrastructure

Most simple baselines in this section do not require feature preprocessing. Dummy classifiers ignore the feature matrix, and the rule-based classifier works directly with a small number of original categorical variables.

Nevertheless, reusable preprocessing objects are introduced here because later model sections depend on them. Two broad preprocessing variants are prepared:

- a scaled preprocessor for scale-sensitive models such as logistic regression, k-nearest neighbours, support vector machines, and neural networks;
- an unscaled preprocessor for tree-based models, where numeric scaling is usually unnecessary.

Categorical variables are one-hot encoded in standard modelling pipelines. Structural categories such as `No internet service` and `No phone service` are treated as valid categories, not as missing values.

4.12 Simple baseline definitions

The majority-class baseline always predicts the most frequent class observed in the training fold:

$$\hat{y} = \arg \max_{c \in \{0,1\}} \sum_{i=1}^n \mathbb{1}\{y_i = c\}.$$

In this dataset, this means always predicting no churn.

The prior-probability baseline uses the empirical class probabilities:

$$\hat{p}(Y = c) = \frac{1}{n} \sum_{i=1}^n \mathbb{1}\{y_i = c\}.$$

Its hard class prediction is still the majority class, but its predicted probabilities correspond to the class prior.

The stratified random baseline randomly samples labels according to the training-fold class distribution:

$$\hat{Y} \sim \text{Categorical}(\hat{p}(Y = 0), \hat{p}(Y = 1)).$$

The uniform random baseline samples labels uniformly:

$$\hat{Y} \sim \text{Categorical}(0.5, 0.5).$$

The exploratory-analysis-inspired rule baseline assigns one risk point for each high-risk condition:

$$s(x) = \sum_{j=1}^m \mathbb{1}\{\text{condition } j \text{ is true}\}.$$

The predicted class is:

$$\hat{y} = \mathbb{1}\{s(x) \geq t\}.$$

The rule used here sets $t = 2$, with the following high-risk conditions:

- month-to-month contract;
- electronic check payment;
- fiber optic internet service;
- no online security;
- no tech support.

This rule is intentionally simple. It is not a final model. Its purpose is to test whether the strongest training-set EDA patterns already contain predictive signal.

4.13 Simple baseline results

Table 25 reports the cross-validated metrics for the simple baselines. Table 26 reports the corresponding confusion-matrix counts in the positive-first order TP, FN, FP, TN .

Table 25: Cross-validated simple-baseline metrics on the training set.

Model	Acc.	Bal. acc.	Prec.	Recall	Spec.	F_1	ROC-AUC	PR-AUC	Pred. pos.	Obs. pos.
EDA-inspired rule	0.6076	0.7032	0.3956	0.9070	0.4994	0.5509	0.8109	0.5435	0.6084	0.2654
Dummy: stratified random	0.6140	0.5065	0.2748	0.2776	0.7354	0.2762	0.5065	0.2680	0.2680	0.2654
Dummy: uniform random	0.5051	0.5057	0.2698	0.5070	0.5045	0.3522	0.5000	0.2654	0.4986	0.2654
Dummy: majority class	0.7346	0.5000	0.0000	0.0000	1.0000	0.0000	0.5000	0.2654	0.0000	0.2654
Dummy: prior probability	0.7346	0.5000	0.0000	0.0000	1.0000	0.0000	0.4999	0.2653	0.0000	0.2654

Table 26: Cross-validated confusion-matrix counts for the simple baselines, shown in positive-first order.

Model	TP	FN	FP	TN
EDA-inspired rule	1356	139	2072	2067
Dummy: stratified random	415	1080	1095	3044
Dummy: uniform random	758	737	2051	2088
Dummy: majority class	0	1495	0	4139
Dummy: prior probability	0	1495	0	4139

4.14 Interpretation

The majority-class and prior-probability baselines have the highest ordinary accuracy among the simple baselines, 0.7346. However, this accuracy is misleading. Both baselines predict no churn for every customer. Therefore, they classify all 4139 non-churners correctly but miss all 1495 churners. Their recall is exactly 0.0000, and their balanced accuracy is only 0.5000.

The stratified random baseline predicts churn at approximately the observed training-set churn rate. Its predicted positive rate is 0.2680, close to the observed positive rate 0.2654. Its balanced accuracy is 0.5065, which is only slightly above random-level performance. This is expected because the predictions are random and not informed by the features.

The uniform random baseline predicts churn for about half of the customers. This gives recall 0.5070, higher than the majority-class baseline, but this is achieved by producing many false positives. The model incorrectly flags 2051 non-churners as churners. Its ROC-AUC is 0.5000, as expected for a random classifier.

The EDA-inspired rule is the only simple baseline that uses feature information. It detects 1356 of the 1495 churners, giving recall 0.9070. This confirms that the strongest EDA patterns contain substantial predictive signal. However, it also incorrectly flags 2072 non-churners, so its specificity is only 0.4994 and its precision is 0.3956.

This makes the EDA-inspired rule a high-recall, low-specificity baseline. It is useful because it shows that simple churn-risk conditions already recover most churners. It is not sufficient as a final model because it predicts churn for about 60.84% of customers, much higher than the observed churn rate of 26.54%. A practical learned model should aim to keep much of this recall while improving precision and specificity.

4.15 Implications for later models

The simple baselines establish the minimum standard for later learned models. A useful learned model should:

- clearly outperform dummy baselines;
- avoid relying only on ordinary accuracy;
- improve balanced accuracy relative to random baselines;
- detect churners with non-trivial recall;
- improve precision and specificity relative to the broad EDA rule;
- provide useful ranking performance through ROC-AUC and PR-AUC;
- eventually support threshold tuning and calibration analysis.

The next modelling stage begins the learned-model sequence with linear classification, least-squares classification, logistic regression, and regularized logistic regression.

5 Linear classification and logistic regression

5.1 Purpose of this stage

This stage introduces the first learned classification models in the project. The previous stage established the evaluation framework, simple dummy baselines, and an EDA-inspired rule. The purpose here is to move from manually specified or non-informative baselines to learned linear classifiers that estimate decision rules from the training data.

The held-out test set is not used in this section. All performance estimates are based on stratified cross-validation inside the training set. Preprocessing is included inside each modelling pipeline, so scaling and one-hot encoding are fitted only on the training portion of each cross-validation fold.

The models considered in this section are regularized least-squares classification, logistic regression with L2 regularization, logistic regression with L1 regularization, and a class-weighted logistic regression variant. These models are useful first learned models because they are mathematically transparent, computationally efficient, and interpretable through their fitted coefficients.

5.2 Linear score functions and decision boundaries

A linear classifier begins with a real-valued score

$$f(x) = w^\top x + b,$$

where $x \in \mathbb{R}^p$ is a feature vector, $w \in \mathbb{R}^p$ is a vector of weights, and $b \in \mathbb{R}$ is an intercept. The score is positive on one side of the decision boundary and negative on the other side.

The decision boundary is the set of feature values for which the score equals zero:

$$w^\top x + b = 0.$$

In two dimensions this boundary is a line, in three dimensions it is a plane, and in higher dimensions it is a hyperplane. A simple binary prediction rule is

$$\hat{y} = \begin{cases} 1, & f(x) \geq 0, \\ 0, & f(x) < 0. \end{cases}$$

In this project, 1 denotes churn and 0 denotes no churn. Therefore, a positive score assigns a customer to the churn side of the fitted linear boundary.

Linear classifiers are limited because they are additive in the transformed feature representation. After one-hot encoding, they can learn separate contributions from contract type, internet service, billing, and numeric variables, but they do not automatically learn complex interactions unless those interactions are explicitly added as features. This limitation is useful to remember when later comparing linear models with trees, ensembles, and neural networks.

5.3 Least-squares classification

Least-squares classification treats classification as a regression-like problem. If labels are encoded as $y_i^* \in \{-1, +1\}$, the fitted score is obtained by minimizing

$$\min_{w,b} \sum_{i=1}^n \left(w^\top x_i + b - y_i^* \right)^2.$$

In matrix form, with the intercept included in the design matrix, this is

$$\min_{\beta} \|X\beta - y^{\star}\|_2^2.$$

A regularized version adds an L2 penalty:

$$\min_{\beta} \|X\beta - y^{\star}\|_2^2 + \alpha\|\beta\|_2^2.$$

This is the idea behind the `RidgeClassifier` used in this section. It provides a linear classification baseline connected to least-squares learning, while the ridge penalty stabilizes fitting after one-hot encoding.

Least-squares classification is not a probability model. It can produce a useful linear score, but it does not directly model $P(Y = 1 \mid X = x)$. Logistic regression is more appropriate when probability estimates and threshold analysis are needed.

5.4 Logistic regression

Logistic regression uses the same linear score,

$$z_i = w^{\top} x_i + b,$$

but maps it to a probability through the sigmoid function:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

The fitted churn probability is

$$\hat{p}_i = P(Y_i = 1 \mid X_i = x_i) = \sigma(w^{\top} x_i + b).$$

The default class prediction uses threshold 0.5:

$$\hat{y}_i = \mathbb{I}\{\hat{p}_i \geq 0.5\}.$$

Because the sigmoid is monotonic and $\sigma(0) = 0.5$, this default threshold corresponds to the linear boundary $w^{\top} x + b = 0$.

Logistic regression can also be written as a log-odds model:

$$\log\left(\frac{p(x)}{1 - p(x)}\right) = w^{\top} x + b.$$

A coefficient w_j is therefore a change in log-odds associated with a one-unit increase in the transformed feature x_j , holding other transformed features fixed. The odds ratio is

$$\exp(w_j).$$

For standardized numeric variables, a one-unit change corresponds to a one-standard-deviation increase in the original numeric feature. For one-hot encoded variables, coefficients describe the contribution of the corresponding category indicator within the fitted linear model. These are fitted associations, not causal effects.

5.5 Likelihood and log loss

For binary outcomes, logistic regression assumes

$$Y_i \mid X_i = x_i \sim \text{Bernoulli}(\hat{p}_i).$$

The probability of observing y_i is

$$P(Y_i = y_i \mid x_i) = \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1-y_i}.$$

The log-likelihood is

$$\ell(w, b) = \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)].$$

Equivalently, logistic regression minimizes the negative log-likelihood,

$$\mathcal{L}(w, b) = - \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)],$$

also called binary cross-entropy or log loss. This loss strongly penalizes confident wrong predictions. For example, assigning a very low probability to a true churner gives a large loss contribution.

5.6 Regularization

Regularization controls model complexity by penalizing large coefficients. L2-regularized logistic regression minimizes

$$\mathcal{L}_{L2}(w, b) = \mathcal{L}(w, b) + \lambda \|w\|_2^2,$$

where

$$\|w\|_2^2 = \sum_{j=1}^p w_j^2.$$

L2 regularization shrinks coefficients toward zero but usually does not set them exactly to zero.

L1-regularized logistic regression minimizes

$$\mathcal{L}_{L1}(w, b) = \mathcal{L}(w, b) + \lambda \|w\|_1,$$

where

$$\|w\|_1 = \sum_{j=1}^p |w_j|.$$

L1 regularization can set coefficients exactly to zero, making it useful for sparse linear models and feature selection.

In scikit-learn, logistic regression uses C as inverse regularization strength. Smaller C means stronger regularization, while larger C means weaker regularization:

$$C \propto \frac{1}{\lambda}.$$

For this first learned-model section, a small log-scale grid is sufficient and more transparent than automated hyperparameter optimization.

5.7 Model comparison

Table 27 reports the cross-validated performance of the main linear classifiers. Logistic regression with L1 and L2 regularization performs almost identically. The L1 model obtains the highest PR-AUC, approximately 0.659, while the L2 model obtains ROC-AUC approximately 0.846 and PR-AUC approximately 0.658. The class-weighted L2 logistic regression has a much higher recall, approximately 0.801, but lower precision, approximately 0.517. This is expected because class weighting makes the model more willing to predict the minority churn class.

Model	Acc.	Bal. acc.	Prec.	Recall	Spec.	F1	ROC-AUC	PR-AUC
Logistic L1 C=1	0.802	0.720	0.654	0.543	0.896	0.593	0.846	0.659
Logistic L2 C=1	0.802	0.719	0.652	0.544	0.895	0.593	0.846	0.658
Logistic L2 balanced C=1	0.748	0.765	0.517	0.801	0.729	0.628	0.846	0.657
RidgeClassifier alpha=1	0.801	0.712	0.659	0.521	0.903	0.582	0.838	0.649

Table 27: Cross-validated linear-model comparison on the training set.

The confusion-matrix counts in Table 28 clarify the tradeoff. The standard L2 logistic regression detects 813 churners and misses 682. The class-weighted variant detects 1198 churners but increases false positives from 434 to 1120. Therefore, class weighting improves recall but does so by flagging many more non-churners.

Model	TP	FN	FP	TN
Logistic L1 C=1	812	683	430	3709
Logistic L2 C=1	813	682	434	3705
Logistic L2 balanced C=1	1198	297	1120	3019
RidgeClassifier alpha=1	779	716	403	3736

Table 28: Cross-validated confusion-matrix counts in positive-first order.

5.8 Regularization results

Table 29 and Figure 9 show the L2 regularization path. Very strong regularization, $C = 0.001$, produces a conservative classifier with high precision but low recall. As C increases, recall and F1 improve. The ranking metrics are stable over most of the grid, with ROC-AUC around 0.845 and PR-AUC around 0.658 once $C \geq 0.1$. This indicates that the logistic model is not highly sensitive to the exact L2 regularization strength over the tested range.

C	Acc.	Bal. acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC
0.001	0.777	0.599	0.780	0.220	0.343	0.839	0.651
0.01	0.805	0.711	0.674	0.511	0.581	0.843	0.656
0.1	0.801	0.717	0.652	0.538	0.590	0.845	0.658
1	0.802	0.719	0.652	0.544	0.593	0.846	0.658
10	0.804	0.723	0.656	0.549	0.598	0.846	0.658
100	0.804	0.723	0.655	0.550	0.598	0.846	0.657

Table 29: L2 logistic regression regularization grid.

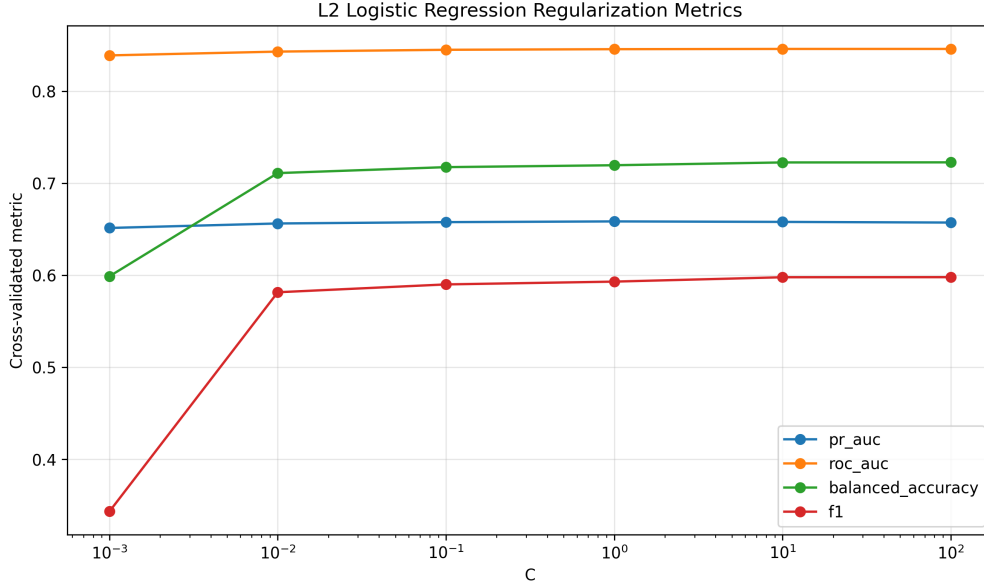


Figure 9: L2 logistic regression metrics over the regularization grid.

The L1 grid in Table 30 and Figure 10 shows a more visible effect of very strong regularization. At $C = 0.001$, the L1 model effectively predicts no churners, so recall and F1 are zero and PR-AUC equals the positive-class prevalence. Once C reaches 0.1 or larger, the L1 model behaves similarly to L2 logistic regression. This suggests that mild L1 regularization is compatible with the predictive structure in the data, but excessive sparsity removes too much useful signal.

C	Acc.	Bal. acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC
0.001	0.735	0.500	0.000	0.000	0.000	0.500	0.265
0.01	0.794	0.673	0.684	0.416	0.517	0.825	0.632
0.1	0.801	0.714	0.655	0.528	0.585	0.844	0.658
1	0.802	0.720	0.654	0.543	0.593	0.846	0.659
10	0.803	0.722	0.654	0.548	0.597	0.846	0.658

Table 30: L1 logistic regression regularization grid.

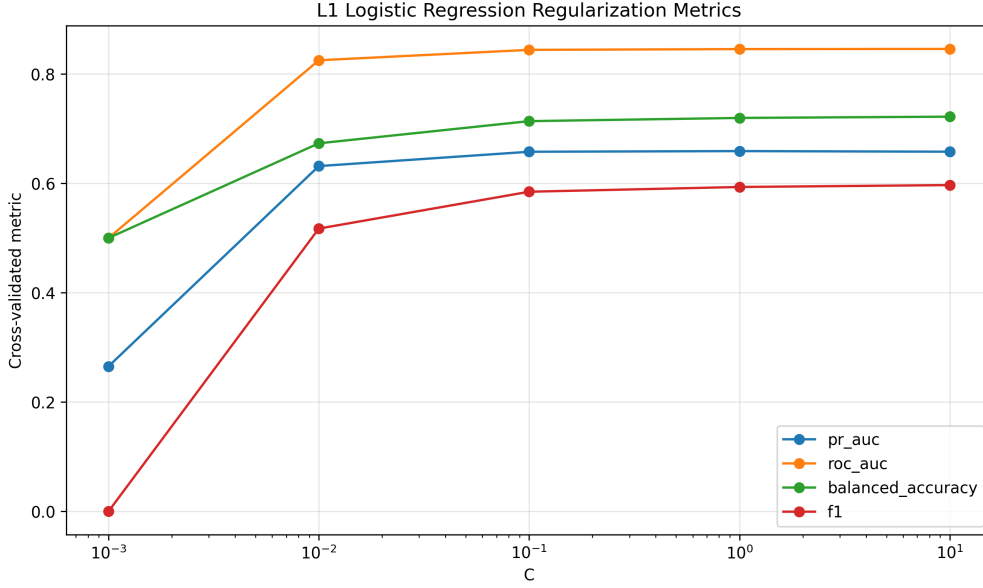


Figure 10: L1 logistic regression metrics over the regularization grid.

5.9 Selected logistic model and coefficient interpretation

For interpretation, the selected model is the L2 logistic regression model with the highest cross-validated PR-AUC in the L2 grid. This selects $C = 1$. The model is then fitted on the full training set only for coefficient extraction. These fitted coefficients are not used to report in-sample performance.

Table 31 and Figure 11 show the largest coefficients by absolute value. The strongest negative coefficient is **tenure**, meaning longer-tenure customers receive lower fitted churn scores. **Contract_Two year** also has a negative coefficient, consistent with the EDA finding that two-year contracts have low churn rates. The strongest positive coefficients are **InternetService_Fiber optic**, **Contract_Month-to-month**, and **TotalCharges**, all associated with higher fitted churn scores in this linear model.

A notable pattern is that **MonthlyCharges** has a negative coefficient while **TotalCharges** has a positive coefficient. This should not be interpreted in isolation as a simple marginal effect. Logistic regression estimates partial associations after controlling for the other encoded features. Since **tenure**, **MonthlyCharges**, and **TotalCharges** are related, their coefficients reflect a conditional linear decomposition rather than the marginal patterns seen in EDA.

Feature	Coef.	Odds ratio	Direction
tenure	-1.255	0.285	lower churn score
Contract_Two year	-0.759	0.468	lower churn score
InternetService_Fiber optic	0.648	1.912	higher churn score
InternetService_DSL	-0.644	0.525	lower churn score
MonthlyCharges	-0.601	0.548	lower churn score
Contract_Month-to-month	0.586	1.797	higher churn score
TotalCharges	0.533	1.703	higher churn score
PaperlessBilling_No	-0.331	0.718	lower churn score
DeviceProtection_No internet service	-0.293	0.746	lower churn score
OnlineBackup_No internet service	-0.293	0.746	lower churn score
OnlineSecurity_No internet service	-0.293	0.746	lower churn score
InternetService_No	-0.293	0.746	lower churn score

Table 31: Largest selected logistic regression coefficients by absolute value.

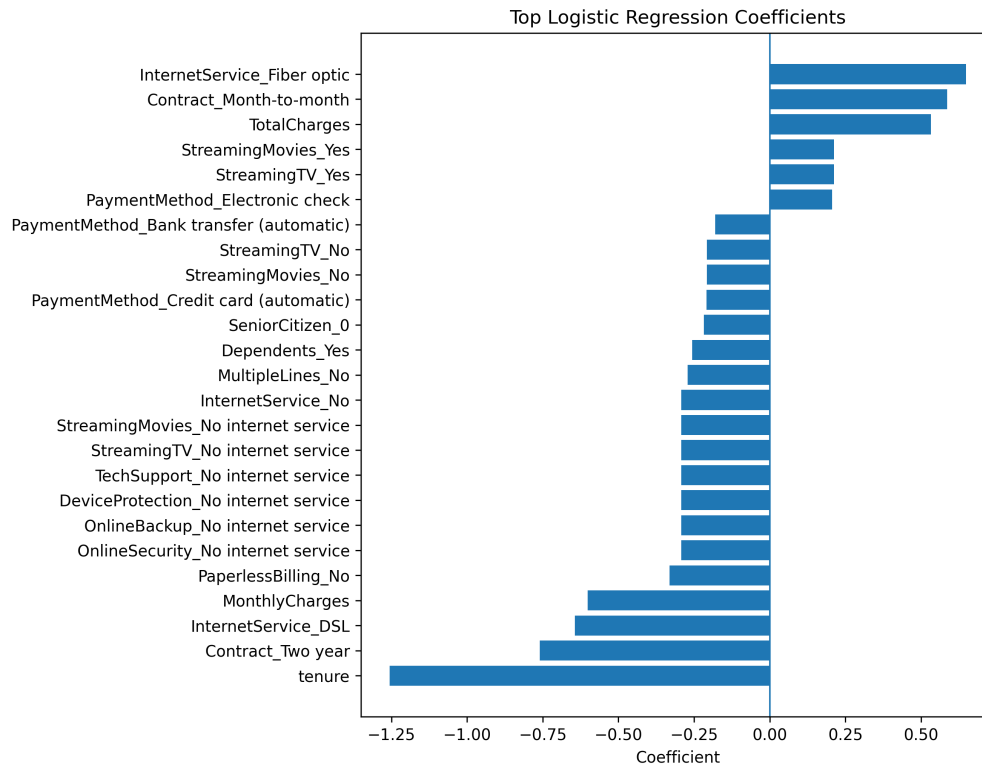


Figure 11: Top logistic regression coefficients by absolute magnitude.

5.10 Threshold tradeoff

The default logistic regression threshold is 0.5, but threshold choice controls the tradeoff between recall, precision, and specificity. Table 32 reports selected thresholds using out-of-fold predicted probabilities. Lowering the threshold increases recall and the predicted positive rate, while raising the threshold increases precision and specificity.

At threshold 0.50, the selected L2 logistic model has precision 0.652, recall 0.544, specificity 0.895, and F1 0.593. At threshold 0.30, recall increases to 0.767, while precision decreases to 0.541 and specificity decreases to 0.765. The F1 score is highest around thresholds 0.30 to 0.35

among the displayed values. This does not mean that the final threshold should be chosen here. Final threshold selection should be postponed until model families are compared and the project objective is specified more explicitly.

Threshold	Acc.	Bal. acc.	Prec.	Recall	Spec.	F1	Pred. pos. rate
0.10	0.611	0.716	0.401	0.942	0.491	0.562	0.624
0.20	0.711	0.757	0.475	0.854	0.659	0.610	0.477
0.25	0.742	0.766	0.509	0.817	0.715	0.627	0.426
0.30	0.766	0.766	0.541	0.767	0.765	0.634	0.376
0.35	0.782	0.761	0.571	0.717	0.805	0.635	0.334
0.40	0.790	0.750	0.594	0.664	0.836	0.627	0.297
0.50	0.802	0.719	0.652	0.544	0.895	0.593	0.221
0.60	0.797	0.669	0.710	0.397	0.942	0.509	0.148
0.70	0.775	0.594	0.779	0.210	0.978	0.331	0.072

Table 32: Threshold tradeoff for selected L2 logistic regression using out-of-fold probabilities.

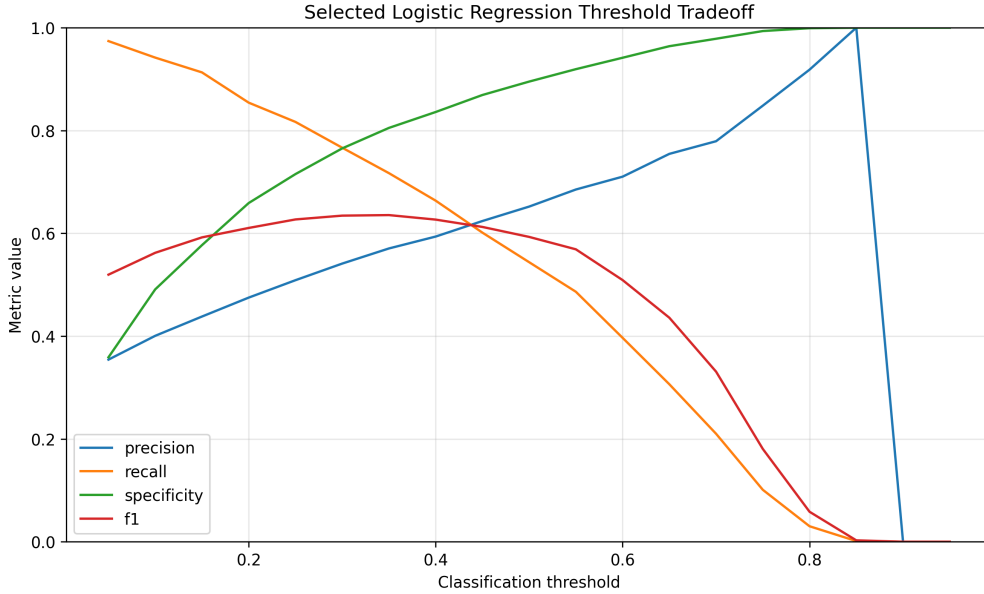


Figure 12: Threshold tradeoff for selected L2 logistic regression.

5.11 ROC and precision-recall curves

Figures 13 and 14 summarize threshold-independent ranking performance for the selected logistic regression model. The ROC curve has AUC approximately 0.846, indicating that the model ranks churners above non-churners substantially better than a random ranking. The precision-recall curve has AUC approximately 0.658, compared with a positive-rate baseline of approximately 0.265. This is important because churn is the minority class, and PR-AUC focuses directly on the quality of positive churn retrieval.

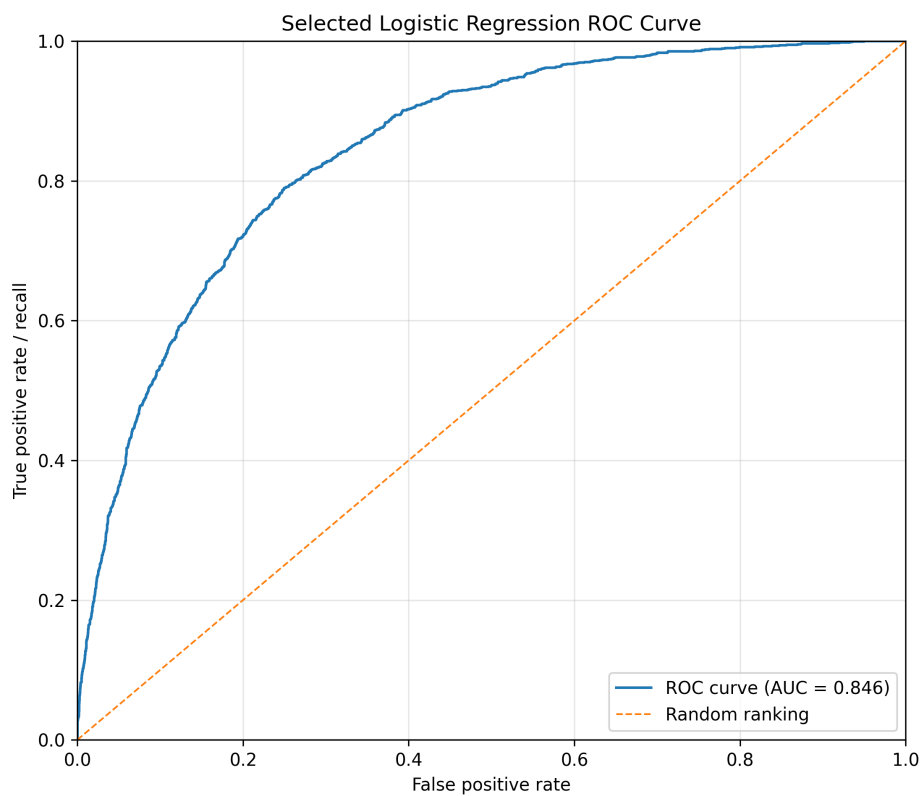


Figure 13: ROC curve for selected L2 logistic regression.

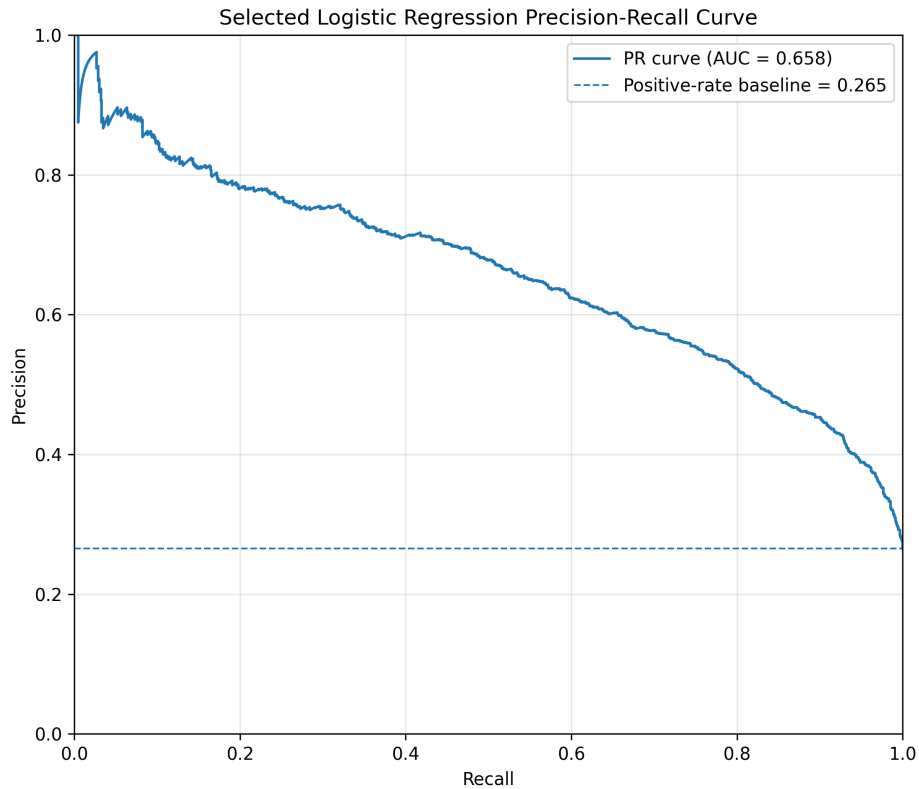


Figure 14: Precision-recall curve for selected L2 logistic regression.

5.12 Summary

Logistic regression is a strong first learned model for the churn task. It clearly improves on non-informative baselines and provides useful probability rankings, with ROC-AUC around 0.846 and PR-AUC around 0.658. L1 and L2 regularization give very similar cross-validated performance once regularization is not excessively strong. Ridge classification is slightly weaker but remains a reasonable least-squares linear baseline.

The class-weighted logistic regression variant demonstrates the central classification tradeoff: it detects many more churners, but it also creates many more false positives. This confirms that model assessment cannot rely on accuracy alone. Confusion-matrix counts, recall, precision, specificity, ROC-AUC, PR-AUC, and threshold behaviour must be considered together.

The coefficient analysis is also useful. Tenure and two-year contracts are associated with lower fitted churn scores, while fiber optic internet, month-to-month contracts, and higher accumulated charges are associated with higher fitted churn scores. These patterns are consistent with the training-set EDA, but they remain predictive associations rather than causal effects.

This section establishes logistic regression as the first interpretable learned benchmark. Later model families should be compared against it by asking whether they improve ranking performance, reduce false positives at useful recall levels, or capture nonlinear feature interactions that linear models cannot represent directly.

6 k-Nearest Neighbours

This section evaluates k-nearest neighbours as the first non-parametric and distance-based classifier in the project. Unlike logistic regression, which estimates a global vector of coefficients, k-nearest neighbours stores the training observations and classifies a new customer by comparing that customer with nearby training customers in the transformed feature space. The model is therefore useful as a contrast to the linear classifiers from Section 5: logistic regression learns global feature weights, whereas k-nearest neighbours relies directly on local similarity.

The same validation discipline is used as in the previous modelling sections. The held-out test set is not used. All reported results are based on stratified cross-validation inside the training set. Preprocessing is kept inside the scikit-learn pipeline so that standardization and one-hot encoding are fitted only on each fold's training data.

6.1 k-nearest-neighbour prediction rule

Let the training set be

$$\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n,$$

where $x_i \in \mathbb{R}^p$ is the transformed feature vector for customer i , and $y_i \in \{0, 1\}$ is the binary churn label. For a new customer with feature vector x , k-nearest neighbours first finds the k training observations closest to x according to a chosen distance function. Let this neighbour set be denoted by

$$\mathcal{N}_k(x).$$

For unweighted binary classification, the estimated churn probability is the fraction of neighbours that churned:

$$\hat{p}(Y = 1 \mid X = x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} y_i.$$

The default hard classification rule at threshold 0.5 is

$$\hat{y} = \mathbb{I}\{\hat{p}(Y = 1 \mid X = x) \geq 0.5\}.$$

Thus, the model predicts churn when churners form at least half of the local neighbourhood.

A distance-weighted variant gives closer neighbours more influence:

$$\hat{p}(Y = 1 \mid X = x) = \frac{\sum_{i \in \mathcal{N}_k(x)} w_i(x) y_i}{\sum_{i \in \mathcal{N}_k(x)} w_i(x)}.$$

In this section, both uniform and distance-based voting are evaluated.

6.2 Distance, scaling, and one-hot encoded features

The distance definition is central to k-nearest neighbours. Two common distances are the Manhattan distance,

$$d_1(x, x') = \sum_{j=1}^p |x_j - x'_j|,$$

and the Euclidean distance,

$$d_2(x, x') = \sqrt{\sum_{j=1}^p (x_j - x'_j)^2}.$$

Both are special cases of the Minkowski distance. In the implementation, Manhattan distance corresponds to $p = 1$, while Euclidean distance corresponds to $p = 2$.

Because k-nearest neighbours computes distances directly from feature values, scaling is essential. Without standardization, variables with larger numeric ranges can dominate the distance calculation. For example, `TotalCharges` has a much larger raw range than `tenure` or `MonthlyCharges`. Therefore, numeric variables are standardized inside the preprocessing pipeline. Categorical variables are one-hot encoded, which allows k-nearest neighbours to operate on mixed tabular data, but also changes the geometry of the feature space. After one-hot encoding, similarity is measured in a higher-dimensional indicator space, which may not always correspond to intuitive customer similarity.

6.3 The role of k

The number of neighbours k controls the smoothness of the classifier. Small values of k produce highly local predictions. For example, when $k = 1$, the prediction copies the label of the single nearest training observation. This can produce low-bias but high-variance decision boundaries that are sensitive to noise.

Larger values of k average over more training observations. This usually increases bias but reduces variance, producing smoother and more stable predictions. If k becomes too large, however, predictions approach the global class distribution and the model can underfit. Therefore, k is a true model-complexity hyperparameter and must be selected using validation data rather than the held-out test set.

6.4 Hyperparameter grid

The k-nearest-neighbour experiment uses a transparent cross-validated grid search. The grid varies the number of neighbours, the voting scheme, and the distance metric:

$$k \in \{1, 3, 5, 7, 11, 15, 21, 31, 51, 75, 101\},$$

$$\text{weights} \in \{\text{uniform}, \text{distance}\},$$

$$p \in \{1, 2\}.$$

The $p = 1$ setting corresponds to Manhattan distance, and $p = 2$ corresponds to Euclidean distance. The selection rule chooses the configuration with the highest cross-validated PR-AUC, using balanced accuracy and F1 as secondary criteria if needed. PR-AUC is used as the primary criterion because churn is the minority class, and the positive-class ranking quality is important.

6.5 Grid-search behaviour

Figure 15 shows the cross-validated PR-AUC across the neighbour grid. Performance improves strongly as k increases from very small values. This pattern indicates that small neighbourhoods are too noisy for this dataset. Larger neighbourhoods smooth over local irregularities and give more stable churn rankings.

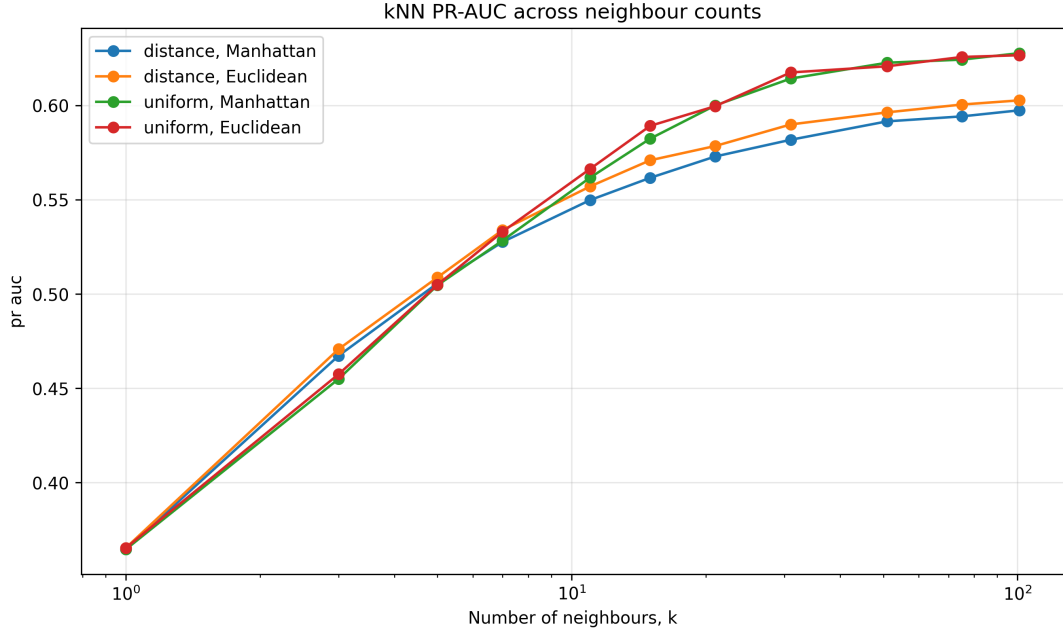


Figure 15: kNN PR-AUC across neighbour counts

Figure 16 shows the same grid from the perspective of balanced accuracy. Balanced accuracy also improves as k increases, but it peaks somewhat earlier than PR-AUC for some configurations. This difference is expected because PR-AUC evaluates ranking quality across thresholds, while balanced accuracy depends on the default hard classification threshold.

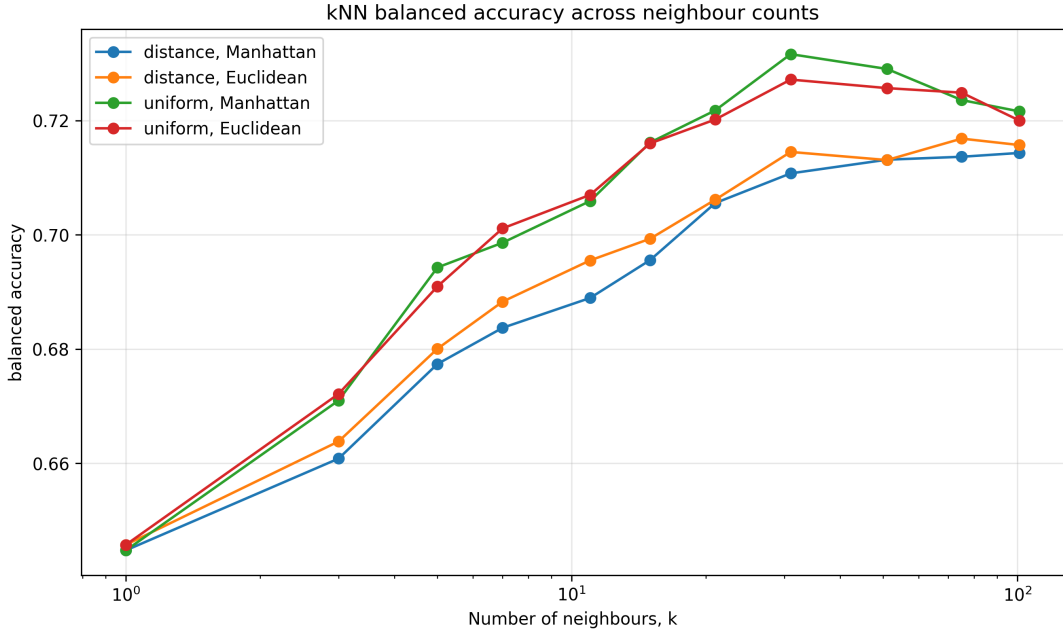


Figure 16: kNN balanced accuracy across neighbour counts

The best configuration by PR-AUC is the uniform-weighted Manhattan model with $k = 101$. Uniform weighting outperforms distance weighting among the best configurations. This suggests that allowing very close neighbours to dominate does not improve generalization in this transformed feature space. A smoother average over many neighbours is more stable.

6.6 Model comparison

Table 33 compares the default $k = 5$ Euclidean kNN model with the selected kNN model. The selected model improves substantially over the default setting. Accuracy rises from approximately 0.765 to 0.792, balanced accuracy from approximately 0.691 to 0.722, ROC-AUC from approximately 0.782 to 0.836, and PR-AUC from approximately 0.505 to 0.628.

Model	Accuracy	Bal. Acc.	Precision	Recall	Specificity	F1	ROC-AUC
Selected kNN, $k = 101$, uniform, Manhattan	0.792	0.722	0.617	0.571	0.872	0.593	0.836
kNN, $k = 5$, uniform, Euclidean	0.765	0.691	0.561	0.532	0.849	0.546	0.782

Table 33: kNN model comparison using cross-validated out-of-fold predictions

The corresponding positive-first confusion-matrix counts are shown in Table 34. The selected model detects more churners and also produces fewer false positives than the default $k = 5$ model. Specifically, the selected model gives 854 true positives and 530 false positives, compared with 796 true positives and 623 false positives for the default model.

Model	TP	FN	FP	TN
Selected kNN, $k = 101$, uniform, Manhattan	854	641	530	3609
kNN, $k = 5$, uniform, Euclidean	796	699	623	3516

Table 34: Positive-first confusion-matrix counts for kNN models

Although the selected kNN model is much better than the default kNN model, it remains weaker than logistic regression from Section 5. The selected logistic regression model had ROC-AUC around 0.846 and PR-AUC around 0.658, while selected kNN has ROC-AUC around 0.836 and PR-AUC around 0.628. This suggests that the global feature-weighting structure learned by logistic regression is more effective than direct distance-based local similarity for this transformed Telco churn dataset.

6.7 Threshold behaviour

The selected kNN model provides predicted probabilities from neighbour proportions. Figure 17 shows how precision, recall, specificity, and F1 change as the classification threshold varies. Lower thresholds identify many more churners but also flag many more non-churners. Higher thresholds improve precision and specificity but miss more churners.

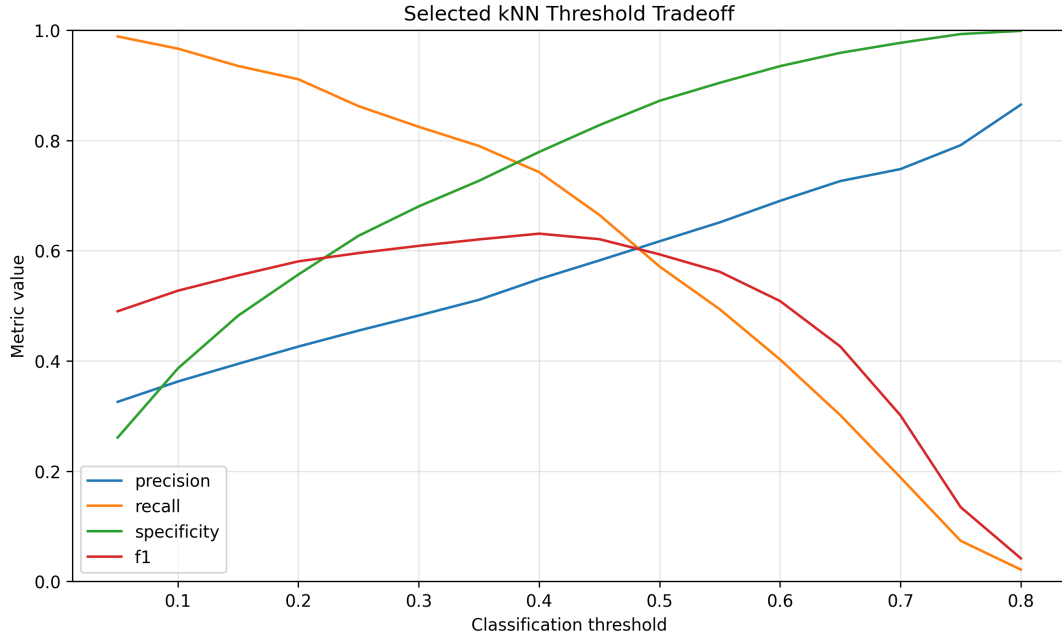


Figure 17: Threshold tradeoff for the selected kNN model

At threshold 0.25, recall is approximately 0.862, but precision is only approximately 0.455. At threshold 0.40, the model reaches a more balanced operating point, with precision approximately 0.548, recall approximately 0.742, specificity approximately 0.779, and F1 approximately 0.631. At the default threshold 0.50, precision improves to approximately 0.617, but recall falls to approximately 0.571. This again shows that threshold 0.50 is not necessarily the best operating point for churn detection, although final threshold selection is postponed until later model comparison.

6.8 ROC and precision-recall curves

Figure 18 shows the ROC curve for the selected kNN model. The ROC-AUC is approximately 0.836, clearly above the random-ranking line. This indicates that the selected kNN model ranks churners above non-churners much better than chance.

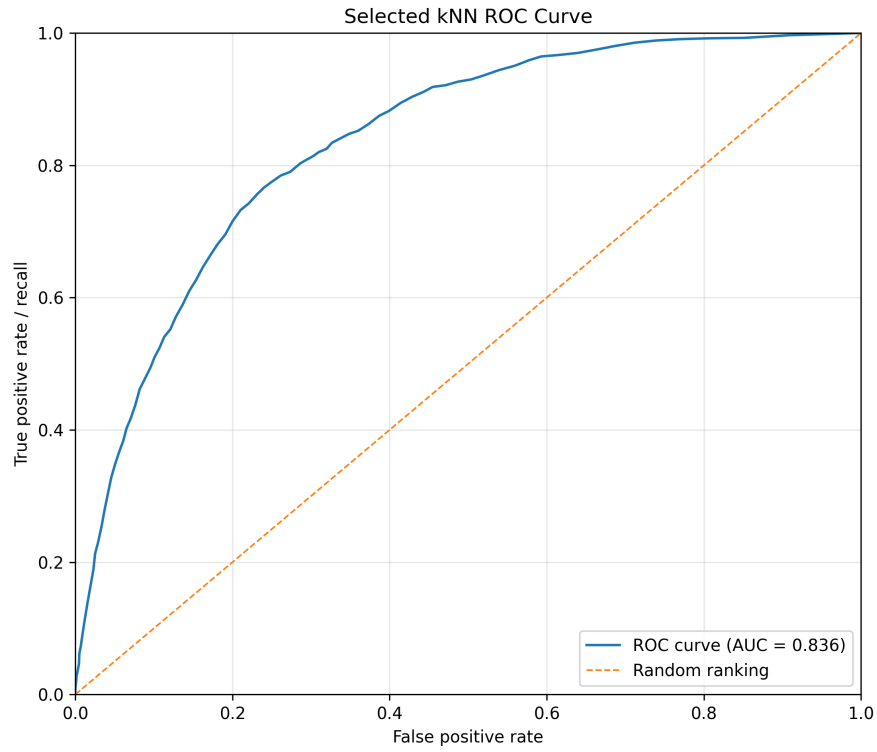


Figure 18: ROC curve for the selected kNN model

Figure 19 shows the precision-recall curve. The PR-AUC is approximately 0.631, well above the positive-rate baseline of approximately 0.265. This confirms that kNN has useful positive-class retrieval ability. However, it remains below the logistic regression PR-AUC from Section 5. The gap is not extremely large, but it is consistent across the ranking metrics.

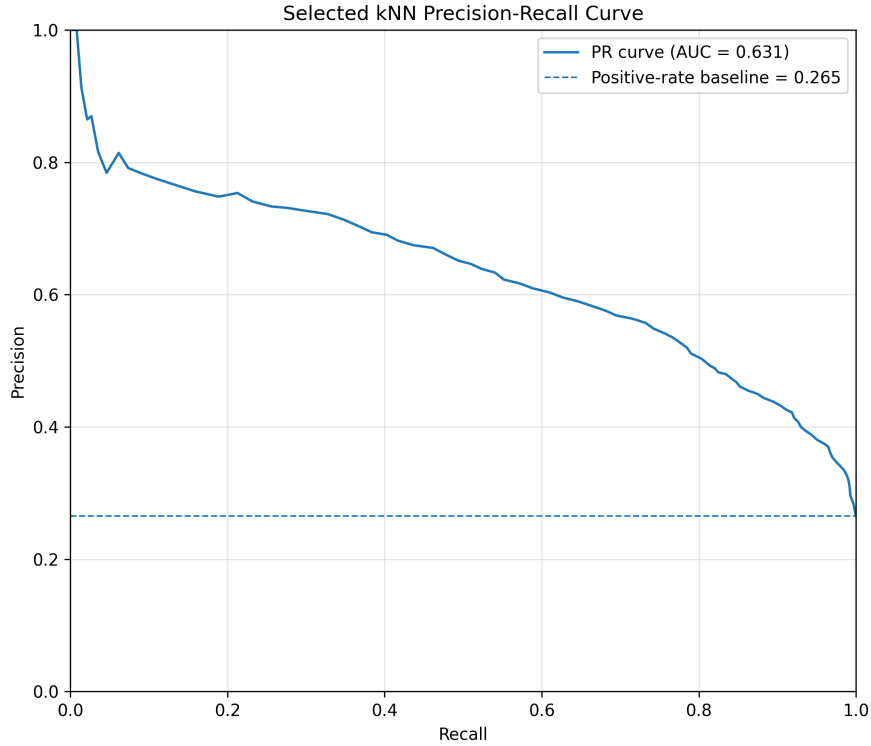


Figure 19: Precision-recall curve for the selected kNN model

6.9 Summary

The k-nearest-neighbour experiment shows that distance-based classification learns meaningful churn structure, but it does not outperform logistic regression. The most important findings are as follows. First, kNN performance is highly sensitive to the number of neighbours. Small k values are too noisy, while larger neighbourhoods perform better. Second, uniform weighting works better than distance weighting in the strongest configurations. Third, the best PR-AUC is obtained by the uniform Manhattan model with $k = 101$, which is a relatively smooth kNN classifier. Fourth, the selected kNN model improves substantially over the default $k = 5$ baseline, but remains weaker than logistic regression in ROC-AUC and PR-AUC.

The section is useful because it introduces local, distance-based learning and the bias-variance role of k . It also illustrates a limitation of distance-based methods on one-hot encoded tabular data: the constructed distance space may not be as informative as a model that learns feature weights directly. The next modelling stage can introduce a different probabilistic classifier, such as Naive Bayes, before moving to tree-based models.

References

- [1] Vrije Universiteit Amsterdam Machine Learning course notes. *Data pre-processing, Part 1: Missing values and outliers*. Used for the principles of not taking data at face value, distinguishing missing feature values from missing target labels, considering the production setting, and distinguishing corrupted values from natural extreme observations.
- [2] Vrije Universiteit Amsterdam Machine Learning course notes. *Model Evaluation, Part 1: Experiments*. Used for the principle that the held-out test set should not be reused during model selection, preprocessing design, feature engineering, or hyperparameter tuning.